Fincon REST API Server

Version: 5.285.002
© Fincon Information Technologies (Pty) Ltd

Requires Fincon Version 5.285.002 or higher

Index:

1. What's new
2. API Request and Response Format
3. Configuration
4. Connecting to the server
5. General principles and list type functions and in general
6. Debtors
7. Creditors
8. Sales Reps
9. Stock Items
10. Serial Numbers
11. Quotations
12. Sales Orders
13. Purchase Orders
14. Production Orders
15. Stock Transfers
16. Customer Repair Orders
17. Returned Merchandise Authorizations
18. Documents
19. API Functions
20. Functions for Mobile Applications

1: What's New

Version 285.001-002:

- SalesRepRecord: The *Email* field was added.
- PurchaseOrderRecord: The *Cheque* field was added.
- SalesOrderRecord and QuotationRecord: The *SupRebate* field was added. **(Refer to the GetAccountPrice function)**
- GetAccountPrice *(New)*: Retrieve the price detail for a stock item and an account. (Included in the functions for stock items) - Version 5.285.002 implements supplier rebates per account.

Version 284.001:

- SalesOrderRecord: The *Assembly* field was added.
- StockRecord: The *ProMaxQuantity* field was added.
- CreateSalesOrderPayment *(New)*: Create receipts allocated to a sales order.

Version 5.283.001-002:

- Updated to accommodate the increase in bill of material lines.
- **UpdateDocumentURL: The parameters have changed**
- GetCROListMobile *(Revised)*: Parameter added for repair order type.
- GetCRODelNotePdfMobile *(Revised)*: Customer Support Calls added.
- GetRepairOrdersMobile *(Revised)*: Customer Support Calls added.
- GetInvoicePdfMobile *(Revised)*: Data updates changed.
- GetQuotePdfMobile *(Revised)*: Data updates changed.
- GetTransferPdfMobile *(Revised)*: Data updates changed.
- QuoteType has been added to QuotationRecord.

Version 5.282.001:

- Login: Field *APIVersion* containing FinconRESTServer's version has been added.
- GetGiftCardInfo *(New)*: Retrieve the balance for a gift card.
- CreateSalesOrder: *SalesOrderGiftCard* for a gift card payment was added.
- GetStockMobile *(Revised)*: Get a stock list filtered by categories or groups.
- GetCROListMobile *(Revised)*: Get list of outstanding CROs per technician code.
- GetCROInfoMobile *(Revised)*: Get the CRO info for a CRO and serial number.
- UpdateCROMobile *(Revised)*: Update selected fields on the CRO.
- CloseCROMobile *(New)*: Set a CRO's status to closed.
- GetCRODelNotePdfMobile *(Replacement)*: Replaces GetCRODelNoteMobile to request (and save if signed) a workshop delivery note in pdf format.
- GetInvoicesMobile *(New)*: Retrieve a list of invoices for signatures.
- GetQuotesMobile *(New)*: Retrieve a list of quotations for signatures.
- GetTransfersMobile *(New)*: Retrieve a list of consignment transfers for signatures.
- GetRepairOrdersMobile *(New)*: Retrieve a list of CROs for signatures.
- GetInvoiceInfoMobile *(New)*: Retrieve information pertaining to a single invoice.
- GetQuoteInfoMobile *(New)*: Retrieve information pertaining to a single quotation.
- GetTransferInfoMobile *(New)*: Retrieve information pertaining to a single consignment transfer.
- GetRepairOrderInfoMobile *(New)*: Retrieve information pertaining to a single CRO.
- GetInvoicePdfMobile *(New)*: Retrieve and save the signed document.
- GetQuotePdfMobile *(New)*: Retrieve and save the signed document.
- GetTransferPdfMobile *(New)*: Retrieve and save the signed document.

Version 5.280.001:

- GetStock, GetStockChanged, CreateStockItem: The number of properties was increased to 20 per stock item.

Version 5.279.001:

- GetDebtorTransactionsPerCustRef *(New)*: Get debtor transactions with the customer reference as filter.
- CRORecord: Fields *WSLocNo* and *WSLocDescrip* have been added.
- GetCROsByLocation *(New)*: Retrieve a list of CROs for a workshop location.
- GetWorkshopLocations *(New)*: Retrieve a list of all the workshop locations.
- ListSalesOrdersByDate, Rep, AccNo and OrderNo: The option to list the last *Count* records has been added. Refer to the *Sales Orders* section for more information.

Version 5.277.001-006:

- The issue with some mobile devices not being able to open Fincon generated pdf files has been resolved.
- SalesOrderRecord: Field *InvLayoutName* has been added.
- GetDebtorCreditInfo *(New)*: Get the credit information for a debtor account.
- GetTerms *(New)*: Retrieve a list of term codes.
- GetStockMobile *(New)*: Get a stock list filtered by categories or groups.
- GetCROListMobile *(New)*: Get list of outstanding CROs per technician code.
- GetCROInfoMobile *(New)*: Get the CRO info for a CRO and serial number.
- UpdateCROMobile *(New)*: Update selected fields on the CRO.
- GetCRODelNoteMobile *(New)*: Request a workshop delivery note in pdf format for a CRO and serial number.
- ***Control characters, specifically decimal 8 (BS), 9 (TAB), 10 (LF), 11 (VT) and 13 (CR), are stripped from incoming fields containing names, addresses, descriptions (including fault, work done, diagnosis, rejected etcetera), instructions and any notes. This is applicable to all the create and update functions.***

Version 5.275.001-002:

- GetStock, GetStockChanged, CreateStockItem: The change to the order in which stock item barcodes are retrieved has been implemented. *From Fincon: The sequence of stock item barcodes is now preserved and the first barcode entered in the barcode list will always be considered to be the preferred barcode which will be used for printing and reports.* The Barcode field in StockRecord will therefore now always contain the preferred barcode and the AdditionalBarcodes array will contain the additional barcodes in the order that they appear in Fincon.
- GetStock, GetStockChanged, CreateStockItem: The minimum quantity (ProQuantity) to qualify for a promotional price has been added.
- CreatePurchaseOrder *(New)*: To create purchase orders.

Version 5.274.001-004

- GetStock, GetStockChanged, CreateStockItem: POMultiples (Multiples of Minimum Purchase Order Quantity) has been added.
- GetStock and GetStockChanged now return additional barcodes too. The first barcode will however still be in the StockRecord's Barcode field.
- GetCancelledSalesOrdersByDate, -Rep, -AccNo and -OrderNo *(New)*: Retrieve a list of cancelled sales orders.
- Customer Repair Orders: The fault, diagnosis, work done, notes and rejected fields have been extended to 300 characters.
- Internal file handling enhancements.

Version 5.273.003

- CreateCROOpt2 *(New)*: This function has been added (in addition to CreateCRO) with revised parameters in order to avoid having to add all the parameters as {"_parameters":[]} in the body.

- GetStockQuantitiesForOpt2 *(New)*: This function has been added (in addition to GetStockQuantitiesFor) with revised parameters in order to avoid having to add all the parameters as {"_parameters":[]} in the body.
- CancelSalesOrder *(New)*: To cancel a sales order. Allocated serial numbers and picked quantities can be released provided that the appropriate rights have been added to the DataID profile.
- GetStock, GetStockChanged, CreateStockItem: Stock item property descriptions and values have been added.

2: API Request and Response Format

Methods:

POST must be used where there is one or more JSON values in the parameter list or when the parameters are passed as {"_parameters":[]} in the body. Otherwise, GET can be used.

URL and request:

The URL consists of the server + datasnap/rest/FinconAPI + "FunctionName". Using the Login function with the following values

Server: http://localhost:4090
Fincon Data ID: Main
Fincon User Name: PASSME
Fincon Password: PASS123
Fincon Use Alternate Extension: False

can be sent as:

GET: http://localhost:4090/datasnap/rest/FinconAPI/"Login"/main/passme/pass123/0/

or

POST: http://localhost:4090/datasnap/rest/FinconAPI/"Login" with {"_parameters":["main","passme","pass123",false]} in the body.

NOTE: The parameters are not case sensitive.

Response:

FinconRESTServer is capable of returning an array of results. The response is therefore always returned in the first element of the "result" array:

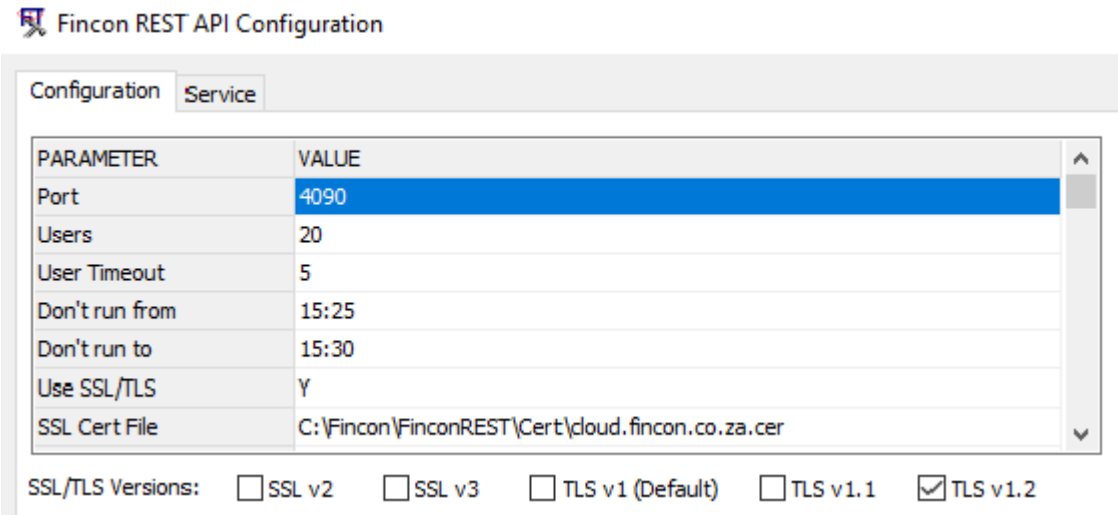
{"result":[{"Connected":true,"ConnectID":"0.2","ErrorInfo":""}]}

If "ErrorInfo" is not blank, the request was unsuccessful and the value of "ErrorInfo" will contain information about the error that occurred.

Refer to **FinconRESTClient** for detailed request and response examples of all the functions.

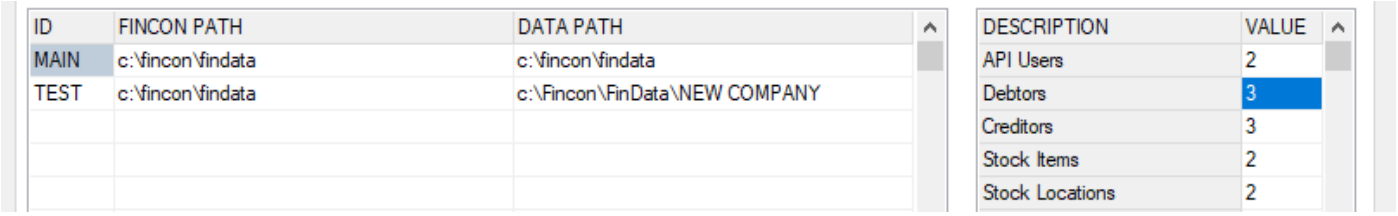
3: Configuration

Server:

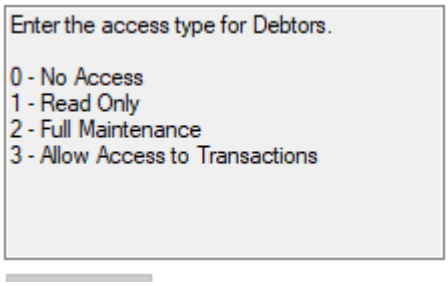


1. **Port:** Enter the HTTP port number to be used.
2. **Users:** Enter the maximum number of concurrent users.
3. **User Timeout:** Enter the user (ConnectID) timeout in minutes. The user timeout per slot is reset with each request. However user slots will automatically be reassigned when they exceed the timeout limit, so this timeout must allow for the execution of the request as well as the transfer of the data to prevent *Invalid connect ID* errors on subsequent requests.
4. **Don't run from:** Enter the time to stop processing requests, normally used to allow for backups or other maintenance.
5. **Don't run to:** Enter the time to start processing requests again.
6. **Use SSL/TLS:** Enter Y to enable security protocols.
7. **SSL Cert File:** Supply (or click Browse to select) the name of the certificate file.
8. **SSL Root Cert File:** Supply (or click Browse to select) the name of the root certificate file.
9. **SSL Key File:** Supply (or click Browse to select) the name of the key file.
10. **Server Username:** Enter the username required to establish a http(s) connection to FinconRESTServer.
11. **Server Password:** Enter the password required to establish a http(s) connection to FinconRESTServer.

Users:



1. **ID:** Enter a profile ID name – this name is required by the Login function to determine the dataset to be used as well as the access rights for the profile.
2. **Fincon Path:** The path to the main company.
3. **Data Path:** The path to the data company which can be a main or sub company.
4. **Access Rights:** Select the access type per category for the profile. The right selected always include the 'lower' rights too. More information regarding the access value is displayed in the hint box situated in the top right area of the form:



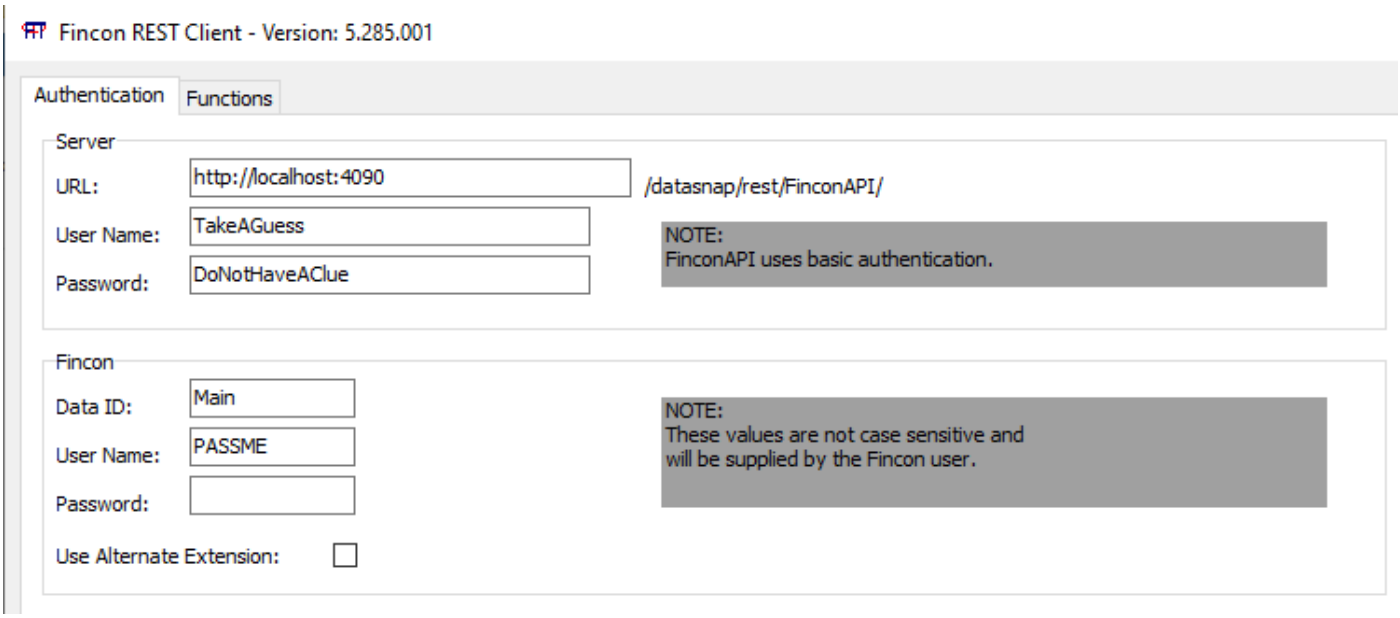
FinconRESTServer can be run as a normal application or as a service. Refer to the *Service* tab for installing, starting and stopping the service.

NOTES:

- **The service must be stopped and restarted when changes were made to the configuration.**
- **FinconRESTConfig must be run with elevated privileges (Run as Administrator) to gain access to the service functions.**

4: Connecting to the server

The URL path is: http(s)://*hostname*:*port*/datasnap/rest/FinconAPI/*function(parameters)*



1. **URL:** The host name (or host IP address) and port – https will automatically imply SSL/TLS.
2. **Server User & Password:** The server username and password as defined in the configuration.
3. **DataID:** The profile ID name as defined in the configuration.
4. **User Name & Password:** The username and password of a valid Fincon user.
5. **Use Alternate Extension:** Dependent on Fincon data file settings.

To connect to the server, use the Login function:

Login (DataID, UserName, Password: String; UseAltExt: Boolean)

Given the values illustrated in the example above, the path will be:

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"Login"/main/passme//0/

The result received is as follows:

{"Connected":true,"ConnectID":"0.2","APIVersion":"n.nnn.nn","ErrorInfo":""} if successful or

{"Connected":false,"ConnectID":"","APIVersion":"n.nnn.nn","ErrorInfo":"Whatever the error may be"} if unsuccessful.

The ConnectID is needed in all subsequent function calls and must therefore be stored in a variable. Parameters are not case sensitive.

To disconnect from the server, use the Logout function:

Logout (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"Logout"/ConnectID/

The result received is as follows:

{"Disconnected":true,"ErrorInfo":""} if successful or

{"Disconnected":false,"ErrorInfo":"Whatever the error may be"} if unsuccessful.

5: General principles and list type functions

The parameters will always include a minimum value, and/or a maximum value, the record number and the count together with applicable selections if any. **The record number must always be 0 on the first call to a list type function.**

Consider the following scenarios to obtain a list of debtor accounts:

List | Changed | Create | Update | Transactions

MinAccNo: MaxAccNo: RecNo: 174 Count: 0 Get

AccNo	DebName	Addr1	Addr2	Addr3
DEB700	DEBTOR 7 IN THE USA	DON'T KNOW		
DEB800	DEBTOR 8 IN THE UK	11 DOWNING STREET	LONDON	
DEB880	TEST DEBTOR			
DEB900	DEBTOR 9 IN THE USA	SEE "STALLONE" IN	TELEPHONE DIRECTORY	
JOH001	TEST			
KOO001	KOOS KOEKEMOER	ADDRESS 1	ADDRESS 2	ADDRESS 3
KOO002	KOOS KOEKEMOER	ADDRESS 1	ADDRESS 2	ADDRESS 3
KOO003	KOOS KOEKEMOER	ADDRESS 1	ADDRESS 2	ADDRESS 3
KOO004	KOOS KOEKEMOER	ADDRESS 1	ADDRESS 2	ADDRESS 3

All the debtor accounts will be retrieved in one go as count is 0. This is obviously not a good idea if there are a few thousand debtor accounts. You can select to get for example only 10 accounts at a time by using the returned record number in the next call to the function to retrieve the next 10 accounts. The returned record number is always the record number of the last record retrieved and it is used to position the record pointer in order to continue from where it ended the last time. If you do not send the last record number, you will always retrieve the same 10 records.

The number of records returned and the last record number will always be in the result:

{"Count":8,"RecNo":174,"ErrorInfo":"","Accounts":[{....}]}

In this case Count is 8, which is less than the 10 requested. This means that there are no more accounts to retrieve.

To retrieve only one account, Set MinAccNo to the desired account number, for example 'deb700' or 'deb7' (The server will always add the zeros at the end up to the account number length) with a record number of 0 and a count of 1.

MinAccNo of 's' and MaxAccNo of 'szzzzz' will retrieve all the accounts of which the account number starts with 's';

FinconRESTServer will do formatting on the following fields:

1. Account numbers will be right padded with zeros up to the required length. This means that 'DEB1' will become 'DEB100'.
2. Document numbers will be left padded with zeros up to the required length. The leading zeros in document numbers can therefore be excluded. A document number of '123' will automatically become '00000123'.
3. Sales Rep and Technician codes will be left padded with zeros up to the required length.
4. Serial numbers will be left or right (depending on the setting in Fincon) truncated to the required length.
5. Uppercase conversions will be done for all information except for fields like an email address or a URL.

Always limit the number of records to be retrieved in a request to a manageable size by using the min/max and count values.

All dates are in CCYYMMDD and times in HH:MM:SS (24 hour) format.

6: Debtors

DebtorRecord:

Key	Type	Len	Dec	Values	Default	Description
AccNo	C	6	0			Account Number
DebName	C	40	0			Debtor Name
Addr1	C	40	0			Address Line 1
Addr2	C	40	0			Address Line 2
Addr3	C	40	0			Address Line 3
PCode	C	8	0			Postal Code
DelName	C	40	0			Delivery Name
DelAddr1	C	40	0			Delivery Address Line 1
DelAddr2	C	40	0			Delivery Address Line 2
DelAddr3	C	40	0			Delivery Address Line 3
DelAddr4	C	40	0			Delivery Address Line 4
DelPCode	C	8	0			Delivery Postal Code
DelInstruc1	C	40	0			Delivery Instructions Line 1
DelInstruc2	C	40	0			Delivery Instructions Line 2
DelInstruc3	C	40	0			Delivery Instructions Line 3
DelInstruc4	C	40	0			Delivery Instructions Line 4
DelInstruc5	C	40	0			Delivery Instructions Line 5
DelInstruc6	C	40	0			Delivery Instructions Line 6
RegName	C	40	0			Registered Company Name
TelNo	C	25	0			Telephone Number
TelNo1	C	25	0			Additional Telephone Number
FaxNo	C	25	0			Fax Number
Contact	C	30	0			Contact Person
Contact1	C	30	0			Additional Contact Person
TaxNo	C	30	0			Tax Registration Number
CompanyNo	C	30	0			Company Registration Number
Memo	C	25	0			Any Applicable Memo
ParentAccNo	C	6	0			Parent Account Number
Notes	C	40	0			Any Applicable Notes
AreaCode	C	3	0			Area Code
RepCode	C	3	0			Sales Rep Code
TermCode	C	3	0			Term Code
Email	C	100	0			Email Address
Email1	C	100	0			Additional Email Address
Active	C	1	0	Y/N	Y	Account Active
CreditGuarantee	C	1	0	Y/N	N	Credit Guarantee
CategoryCode	C	3	0			Account Category Code
ZeroTax	C	1	0	Y/N	N	Zero Tax Rated
CurrencyCode	C	2	0			Currency Code
DeliveryMethod	C	1	0	C/D/R	C	Delivery Method
FinRef	C	10	0			Reference
AccRef	C	20	0			Account Reference
CreditCover	N	18	2			Credit Cover Amount
CreditLimit	N	18	2			Credit Limit
BankName	C	30	0			Bank Name

BankAccNo	C	30	0			Bank Account Number
Branch	C	8	0			Branch Code
StatementMail	C	100	0			Statement Email Address
StatementPrint	C	1	0	Y/N	Y	Statement Required
AccManager	C	3	0			Account Manager Rep Code
InternalRep	C	3	0			Internal Rep Code
Discount	N	6	2			Discount Percentage
PriceStruc	I	2	0	1-6	1	Price Structure
CurrentBalance	N	18	2			Current Account Balance
SecurityLevel	C	1	0	0-9	0	Security Level
AutoReval	C	1	0	Y/N	N	Auto Revaluation at Period End
DetailChange	C	1	0	Y/N	N	Allow Debtor Detail Change
BackOrders	C	1	0	Y/N	Y	Maintain Back Orders
WebList	C	1	0	Y/N	N	Export Account for ECommerce
OnHold	C	1	0	Y/N	N	Account On Hold
Rebate	N	18	2			Rebate Percentage
Location	C	6	0			Default Stock Location for Orders
SettlementDisc	N	18	2			Settlement Discount Percentage
TargetGP	N	18	2			Target Gross Profit
Password	C	8	0			Password for ECommerce
PriceList	C	1	0	Y/N	N	Price List Active
PriceListExpDate	D	8	0			Price List Expiry Date
ChangeDate	D	8	0			Date Last Changed
ChangeTime	C	8	0			Time Last Changed

GetDebAccounts (ConnectID, MinAccNo, MaxAccNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDebAccounts"/ConnectID/MinAccNo /MaxAccNo/RecNo/Count/

GetDebAccountsByAccRef (ConnectID, MinRefNo, MaxRefNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDebAccountsByAccRef"/ConnectID/MinRefNo /MaxRefNo/RecNo/Count/

GetDebAccountsChanged (ConnectID, FromDate, FromTime: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDebAccountsChanged"/ConnectID/FromDate /FromTime/RecNo/Count/

These functions retrieve a list of debtor accounts according to the function and parameters selected. Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

The result received is as follows:

{"Count":8,"RecNo":174,"ErrorInfo":"","Accounts":[{DebtorRecord}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

CreateDebAccount (ConnectID: String; DebAccRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateDebAccount"/ConnectID/
With `{DebtorRecord}` **in the body.**

DebAccRecord is the DebtorRecord structure listed above. If the AccNo value is blank, the server will assign an account number. The DebName value is mandatory. All the other values will be filled with the default values where applicable. CurrentBalance, ChangeDate and ChangeTime are read only fields. **The CurrencyCode can only be set when an account is created – make sure that it is correct.**

The result received is as follows:

`{"ErrorInfo":"","AccountInfo":{DebtorRecord}}` if successful or

`{"ErrorInfo":"Account name not supplied or whatever the error may be."}` if unsuccessful.

UpdateDebAccount (ConnectID: String; DebAccRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"UpdateDebAccount"/ConnectID/
With `{DebtorRecord}` **in the body.**

Retrieve the full current account information first before updating. Any changes on the currency code will be ignored.

The result received is as follows:

`{"ErrorInfo":"","AccountInfo":{DebtorRecord}}` if successful or

`{"ErrorInfo":"Whatever the error may be."}` if unsuccessful.

DebtorCreditRecord:

Key	Type	Len	Dec	Values	Default	Description
AccNo	C	6	0			Accounts Number
CombinedAccount	C	1	0			The information supplied was calculated on a parent/children account structure
NeedsApproval	C	1	0			Sales Order approval is required
CreditLimit	N	18	2			Credit Limit
AccBalance	N	18	2			Current Account Balance
UDPayments	N	18	2			Unreconciled Payments
OpenOrders	N	18	2			Total Outstanding Sales Orders
ApprovedOrders	N	18	2			Total Approved Sales Orders
Overdue	C	1	0			Overdue Invoices Present
RepCode	C	3	0			Sales Rep Code
TermCode	C	3	0			Term Code
TermType	I	8	0			Term Type
MaxDays	I	8	0			The Maximum Number of Days for payment
CurrencyCode	C	2	0			Currency Code
ExchangeRate	N	18	7			Current Exchange Rate

GetDebtorCreditInfo (ConnectID: String; AccNo: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/" GetDebtorCreditInfo"/ConnectID/AccNo/

AccNo: The account for which the credit info must be retrieved.

The credit information is dependent on the applicable settings in Fincon such as *Include UD Payments, Approve Sales Orders, Use Parent Account and Overdue Payment Checking*.

The result received is as follows:

{"ErrorInfo":"," CreditInfo":{DebtorCreditRecord}} if successful or

{"ErrorInfo":"Invalid account number whatever the error may be."} if unsuccessful.

DebtorTransactionRecord:

Key	Type	Len	Dec	Values	Default	Description
Entry	C	1	0			Entry Type
DocNo	C	8	0			Document Number
OrderNo	C	8	0			Sales Order Number
InvoiceNo	C	8	0			Invoice Number
Date	D	8	0			Transaction Date
AccNo	C	6	0			Account Number
DebName	C	40	0			Debtor Name
DelName	C	40	0			Delivery Name
CustomerRef	C	8	0			Customer Reference
RepCode	C	3	0			Sales Rep Code
Location	C	6	0			Stock Location
CurrencyCode	C	2	0			Currency Code
ExchangeRate	N	18	7			Transaction Exchange Rate
TotalCost	N	18	2			Total Cost
TotalRetail	N	18	2			Total Retail (Excl)

DebtorTransactionDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Quantity
LineCost	N	18	2			Line Cost
LineRetail	N	18	2			Line Retail (Excl)

GetDebtorTransactions (ConnectID, AccNo, MinDate, MaxDate, LocNo, TransType: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDebtorTransactions"/ConnectID/AccNo/ MinDate/MaxDate/LocNo/TransType/RecNo/Count/

and GetDebtorTransactionsPerCustRef (ConnectID, MinCustRef, MaxCustRef, LocNo, TransType: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDebtorTransactionsPerCustRef"/ConnectID/ MinCustRef/MaxCustRef/LocNo/TransType/RecNo/Count/

The transaction file may contain a large number of opening transactions, in which case it is safer to provide an account number or minimum date to prevent the unnecessary traversing of opening transactions.

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

TransType can be blank to include invoices and credit notes, or 'I' or 'C' to include just invoices or just credit notes respectively.

The result received is as follows:

{"Count":1,"RecNo":38,"ErrorInfo":"","DebtorTransactions":[{DebtorTransactionRecord,"TransactionDetail":[{DebtorTransactionDetailRecord}]}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of DebtorTransactionRecord structures that includes an array of DebtorTransactionDetailRecord structures (Key: TransactionDetail) on each transaction record.

The cost and retail amounts are in the local currency. Use the ExchangeRate value to calculate the amount in the debtor account's currency if required.

DebtorPriceListRecord:

Key	Type	Len	Dec	Values	Default	Description
AccNo	C	6	0			Account Number
ItemNo	C	25	0			Item Number
Value	N	18	4			Value
ValueType	C	1	0			Value Type

The debtor price list consists of 2 different types of values:

If ValueType is I, ItemNo is a stock item and the value is the unit price excluding tax. If ValueType is C, ItemNo contains the stock category and Value is the discount percentage for the category.

GetDebtorPriceList (ConnectID, AccNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDebtorPriceList"/ConnectID/AccNo/RecNo/Count/

The result received is as follows:

{"Count":1,"RecNo":38,"ErrorInfo":"","PriceList":[{DebtorPriceListRecord},{DebtorPriceListRecord}...}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

TaxRecord:

Key	Type	Len	Dec	Values	Default	Description
TaxCode	N	6	0			Tax Code
Description	C	30	0			Description
InputPrc	N	18	2			Input Tax Percentage
OutputPrc	N	18	2			Output Tax Percentage

GetTaxCodes (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetTaxCodes"/ConnectID/

The result received is as follows:

{{"Count":3,"ErrorInfo":"," "TaxDetail": [{TaxRecord}]} if successful or

{{"Count":0,"ErrorInfo":"Whatever the error may be."} if unsuccessful.

ExchangeRecord:

Key	Type	Len	Dec	Values	Default	Description
ExCode	C	2	0			Exchange Rate Code
Abbreviation	C	6	0			Currency Abbreviation
Description	C	30	0			Currency Description
Symbol	C	3	0			Currency Symbol
Rate	N	18	7			Current Exchange Rate

GetExchangeRates (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetExchangeRates"/ConnectID/

The result received is as follows:

{{"Count":3,"ErrorInfo":"," "ExchangeDetail": [{ExchangeRecord}]} if successful or

{{"Count":0,"ErrorInfo":"Whatever the error may be."} if unsuccessful.

TermsRecord:

Key	Type	Len	Dec	Values	Default	Description
TermCode	C	3	0			Terms Code
Description	C	30	0			Description
TermType	N	8	0			Term Type (Refer to Fincon)
TermDays	N	8	0			Term Days
Extension	N	8	0			Days Extension

GetTerms (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetTerms"/ConnectID/

The result received is as follows:

{{"Count":3,"ErrorInfo":"," "TermsDetail": [{TermsRecord}]} if successful or

{{"Count":0,"ErrorInfo":"Whatever the error may be."} if unsuccessful.

7: Creditors

CreditorRecord:

Key	Type	Len	Dec	Values	Default	Description
AccNo	C	6	0			Account Number
CredName	C	40	0			Creditor Name
Addr1	C	40	0			Address Line 1
Addr2	C	40	0			Address Line 2
Addr3	C	40	0			Address Line 3
PCode	C	8	0			Postal Code
ColName	C	40	0			Collection Name
ColAddr1	C	40	0			Collection Address Line 1
ColAddr2	C	40	0			Collection Address Line 2
ColAddr3	C	40	0			Collection Address Line 3
ColAddr4	C	40	0			Collection Address Line 4
ColPCode	C	8	0			Collection Postal Code
TelNo	C	25	0			Telephone Number
TelNo1	C	25	0			Additional Telephone Number
FaxNo	C	25	0			Fax Number
Contact	C	30	0			Contact Person
Contact1	C	30	0			Additional Contact Person
TaxNo	C	30	0			Tax Registration Number
Memo	C	25	0			Any Applicable Memo
ParentAccNo	C	6	0			Parent Account Number
Notes	C	40	0			Any Applicable Notes
AreaCode	C	3	0			Area Code
TermCode	C	3	0			Term Code
Email	C	100	0			Email Address
Email1	C	100	0			Additional Email Address
Active	C	1	0	Y/N	Y	Account Active
ImportAcc	C	1	0	Y/N	N	Import Supplier
CurrencyCode	C	2	0			Currency Code
FinRef	C	10	0			Account Reference
AccRef	C	20	0			Reference
CreditLimit	N	18	2			Credit Limit
BankName	C	30	0			Bank Name
BankAccNo	C	30	0			Bank Account Number
Branch	C	8	0			Branch Code
RemittanceMail	C	100	0			Remittance Email Address
RemittancePrint	C	1	0	Y/N	Y	Remittance Required
CurrentBalance	N	18	2			Current Account Balance
SecurityLevel	C	1	0	0-9	0	Security Level
AutoReval	C	1	0	Y/N	N	Auto Revaluation at Period End
DetailChange	C	1	0	Y/N	N	Allow Creditor Detail Change
BackOrders	C	40	1	Y/N	Y	Maintain Back Orders
SettlementDisc	N	18	2			Settlement Discount Percentage
ChangeDate	D	8	0			Date Last Changed
ChangeTime	C	8	0			Time Last Changed

GetCredAccounts (ConnectID, MinAccNo, MaxAccNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCredAccounts"/ConnectID/MinAccNo /MaxAccNo/RecNo/Count/

GetCredAccountsChanged (ConnectID, FromDate, FromTime: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCredAccountsChanged"/ConnectID/FromDate /FromTime/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

The result received is as follows:

{"Count":8,"RecNo":174,"ErrorInfo":"","Accounts":[{CreditorRecord}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

CreateCredAccount (ConnectID: String; CredAccRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateCredAccount"/ConnectID/
With `{CreditorRecord}` **in the body.**

CredAccRecord is the CreditorRecord structure listed above. If the AccNo value is blank, the server will assign an account number. The CredName value is mandatory. All the other values will be filled with the default values where applicable. CurrentBalance, ChangeDate and ChangeTime are read only fields. **The CurrencyCode can only be set when an account is created – make sure that it is correct.**

The result received is as follows:

{"ErrorInfo":"","AccountInfo":{CreditorRecord}} if successful or

{"ErrorInfo":"Account name not supplied or whatever the error may be."} if unsuccessful.

UpdateCredAccount (ConnectID: String; CredAccRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateCredAccount"/ConnectID/
With `{CreditorRecord}` **in the body.**

Retrieve the full current account information first before updating. Any changes on the currency code will be ignored.

The result received is as follows:

{"ErrorInfo":"","AccountInfo":{CreditorRecord}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

CreditorTransactionRecord:

Key	Type	Len	Dec	Values	Default	Description
Entry	C	1	0			Entry Type
DocNo	C	8	0			Document Number
GRVNo	C	8	0			GRV Number
OrderNo	C	8	0			Order Number **
Date	D	8	0			Transaction Date
AccNo	C	6	0			Account Number
CredName	C	40	0			Creditor Name
SupplierInv	C	8	0			Supplier Invoice Number
Location	C	6	0			Stock Location
CurrencyCode	C	2	0			Currency Code
ExchangeRate	N	18	7			Transaction Exchange Rate
TotalCost	N	18	2			Total Cost
TotalFOB	N	18	2			Total FOB Amount

Entry Types:

G – Supplier GRV
M – Import GRV
T – Branch Transfer GRV
U – Head Office GRV
D – Supplier Debit Note
E – Import Debit Note
X – Branch Transfer Debit Note
V – Head Office Debit Note

***** In the case of Import Shipments and Combined Import Shipments the order number in CreditorTransactionRecord is the number of the first Purchase Order inserted on the GRV. Refer to the CreditorTransactionDetailRecord entries to obtain all the relevant Purchase Order numbers.***

CreditorTransactionDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Quantity
LineCost	N	18	2			Line Cost
LineFOB	N	18	2			Line FOB Amount
OrderNo	C	8	0			Order Number

GetCreditorTransactions (ConnectID, AccNo, MinDate, MaxDate, LocNo, TransType: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCreditorTransactions"/ConnectID/AccNo/ MinDate/MaxDate/LocNo/TransType/RecNo/Count/

The transaction file may contain a large number of opening transactions, in which case it is safer to provide an account number or minimum date to prevent the unnecessary traversing of opening transactions.

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

TransType can be blank to include goods received vouchers and debit notes, or ‘G’ or ‘D’ to include just goods received vouchers or just debit notes respectively.

The result received is as follows:

{"Count":1,"RecNo":38,"ErrorInfo":"","CreditorTransactions":[{CreditorTransactionRecord,"TransactionDetail":[{CreditorTransactionDetailRecord}]}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of CreditorTransactionRecord structures that includes an array of CreditorTransactionDetailRecord structures (Key: TransactionDetail) on each transaction record.

The cost amounts are in the local currency and represent the full cost including charges, but excluding tax. The FOB amounts are in the creditor invoice’s currency and represent the original foreign currency amounts as entered on the goods received voucher or debit note and exclude charges and tax.

The purchase order number is listed per line as a goods received voucher can include transactions from more than one purchase order with the charges are shared amongst them. Refer to the Import Shipment function in Fincon.

8: Sales Reps

SalesRepRecord:

Key	Type	Len	Dec	Values	Default	Description
RepCode	C	3	0			Rep Code
RepName	C	15	0			Rep Name
Addr1	C	30	0			Address Line 1
Addr2	C	30	0			Address Line 2
Addr3	C	30	0			Address Line 3
Addr4	C	30	0			Address Line 4
TelNo	C	20	0			Telephone Number
EMail	C	100	0			Email Address
SalesTarget	N	18	0			Sales Target per Period
GPTarget	N	18	0			Gross Profit Target per Period
SalesAcc	C	6	0			Ledger Account for Sales
CostAcc	C	6	0			Ledger Account for COS
UseRepAcc	C	1	0	Y/N	Y	Use Rep's Sales and Cost Acc
Department	C	6	0			Rep's Department
Active	C	1	0	Y/N	Y	Account Active

GetSalesReps (ConnectID, MinRepCode, MaxRepCode: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetSalesReps"/ConnectID/MinRepCode/ MaxRepCode/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

The result received is as follows:

{"Count":8,"RecNo":174,"ErrorInfo":"","Reps":[{SalesRepRecord}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

CreateSalesRep (ConnectID: String; SalesRepRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateSalesRep"/ConnectID/
With `{SalesRepRecord}` **in the body.**

SalesRepRecord is the structure listed above. The RepCode and RepName fields are mandatory. All the other values will be filled with the default values where applicable.

The result received is as follows:

{"ErrorInfo":"","RepInfo":{SalesRepRecord}} if successful or

{"ErrorInfo":"Account name not supplied or whatever the error may be."} if unsuccessful.

UpdateSalesRep (ConnectID: String; SalesRepRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"UpdateSalesRep"/ConnectID/
With `{SalesRepRecord}` **in the body.**

Retrieve the full current sales rep information first before updating.

The result received is as follows:

{"ErrorInfo":"","RepInfo":{SalesRepRecord}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

9: Stock Items

StockRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Description	C	40	0			Item Description
ItemType	C	1	0	0-4	0	ItemType
SubItemType	C	1	0			Sub Item Type
UnitOfMeasure	C	2	0			Unit Of Measure
Category	C	3	0			Item Category
CatDescription	C	30	0			Category Description
CatWebList	C	1	0	Y/N	Y	Category Web List Export
Brand	C	15	0			Item Brand
BrandDescription	C	30	0			Brand Description
Group	C	6	0			Item Group
GroupDescription	C	30	0			Group Description
ItemClass	C	15	0			Category Class
ItemClassDescription	C	30	0			Class Description
SerialNumbers	C	1	0	Y/N	N	Serial Number Tracking
SalesAcc	C	6	0			Sales Ledger Account
CostAcc	C	6	0			Cost of Sales Ledger Account
Warranty	I	6	0			Item Warranty Months
Negative	C	10	0	Y/N	N	Allow Negative Stock
BinLocation	C	25	0			Bin Location (Default)
BinLocation01	C	25	0			Bin Location for Location 01
…through to...						…..
BinLocation40	C	25	0			Bin Location for Location 40
Discount	N	18	2			Item Discount Percentage
Rebate	N	18	2			Item Rebate Amount
LineComms	N	18	2			Line Comms
Notes	C	250	0			Item Notes
PriceList	C	1	0	Y/N	N	Include on Price List
WebList	C	1	0	Y/N	N	Item Web List Export
SecurityLevel	C	1	0	0-9	0	Security Level
SupplierAccNo	C	6	0			Supplier Account Number
SupplierName	C	40	0			Supplier Name
SupplierItemNo	C	25	0			Supplier Item Number
SupplierReference	C	25	0			Supplier Reference
VariableDescription	C	1	0	Y/N	N	Variable Item Description
Active	C	1	0	Y/N	Y	Item Active
UnitCost	N	18	4			Unit Cost
LastPurchaseCost	N	18	2			Last Purchase Cost
PriceStructure	I	8	0			Price Structure
SellingPrice1	N	18	4			Selling Price 1 (Excl)
SellingPrice2	N	18	4			Selling Price 2 (Excl)
SellingPrice3	N	18	4			Selling Price 3 (Excl)
SellingPrice4	N	18	4			Selling Price 4 (Excl)
SellingPrice5	N	18	4			Selling Price 5 (Excl)
SellingPrice6	N	18	4			Selling Price 6 (Excl)

TaxCode	I	8	0	0-24	1	Tax Code
TaxPercentage	N	18	2			Tax Percentage
MinMargin	N	18	2			Minimum Margin Percentage
ProPrice	N	18	4			Promotional Price (Excl)
ProPriceType	C	6	0			Price Type/s to which the price applies
ProFromDate	D	8	0			Promotion Start Date
ProToDate	D	8	0			Promotion Expiry Date
ProQuantity	N	18	2			Promotion Minimum Quantity
ProMaxQuantity	N	18	2			Promotion Maximum Quantity
Barcode	C	20	0			Item Barcode
URL	C	200	0			Item URL
Weight	N	18	4			Item Weight
BoxWidth	N	18	4			Box Width
BoxLength	N	18	4			Box Length
BoxHeight	N	18	4			Box Height
BoxQuantity	N	18	4			Box Quantity
VolWeight	N	18	4			Volumetric Weight per Package
SizeFactor	N	18	4			Size Factor
Comment	C	40	0			Item Comment
MinPurchaseQuantity	I	8	0			Minimum Purchase Order Quantity
POMultiples	C	1	0	Y/N	N	Multiples of Minimum Purchase Order Quantity
MinOrderQuantity	I	8	0			Minimum Sales Order Quantity
Multiples	C	1	0	Y/N	N	Multiples of Minimum Sales Order Quantity
AltDescription	C	40	0			Alternate Item Description
PropDescription01	C	15	0			Property Description 1
…through to...						…..
PropDescription10	C	15	0			Property Description 10
PropValue01	C	30	0			Property Value 1
…through to...						…..
PropValue10	C	30	0			Property Value 10
TechnicianStock	N	18	2			Technician Stock On Hand
WIPStock	N	18	2			WIP Stock On Hand
ConsignmentStock	N	18	2			Consignment Stock On Hand
ChangeDate	D	8	0			Date Last Changed
ChangeTime	C	8	0			Time last Changed

StockLocRecord:

Key	Type	Len	Dec	Values	Default	Description
LocNo	C	6	0			Stock Location
InStock	N	18	2			Normal Stock On Hand
SalesOrders	N	18	2			Sales Order Quantity
PurchaseOrders	N	18	2			Purchase Order Quantity
FaultyStock	N	18	2			Faulty Stock On Hand
SwopoutStock	N	18	2			Swopout Stock On Hand
SupplierStock	N	18	2			Stock at Suppliers

GetStock (ConnectID, MinItemNo, MaxItemNo, LocNo: String; WebOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetStock"/ConnectID/MinItemNo/ MaxItemNo/LocNo/WebOnly/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

LocNo represents the required stock location. If blank, all active locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

WebOnly: Select to only include items that will be exported for Ecommerce purposes.

The result received is as follows:

{"Count":1,"RecNo":88,"ErrorInfo":"","Stock":[{StockRecord,"StockLoc":[{StockLocRecord}], ”AdditionalBarcodes”:[{“Barcode”:“Barcode2”},{“Barcode”:”Barcode3”}…]}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

GetStockChanged (ConnectID, FromDate, FromTime: String, LocNo: String; WebOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetStockChanged”/ConnectID/FromDate/ToDate/FromTime/LocNo/WebOnly/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

The result received is as follows:

{"Count":8,"RecNo":174,"ErrorInfo":"","Stock":[{StockRecord,"StockLoc":[{StockLocRecord}], ”AdditionalBarcodes”:[{“Barcode”:“Barcode2”},{“Barcode”:”Barcode3”}…]}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

StockQtyHeaderRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Description	C	40	0			Item Description
ItemType	C	1	0			Item Type
TechnicianStock	N	18	2			Tecnician Stock On Hand
WIPStock	N	18	2			WIP Stock On Hand
ConsignmentStock	N	18	2			Consignment Stock On Hand

GetStockQuantities (ConnectID, MinItemNo, MaxItemNo, LocNo: String; WebOnly, SkipIfZero: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetStockQuantities”/ConnectID/MinItemNo/MaxItemNo/LocNo/WebOnly/SkipIfZero/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

LocNo represents the required stock location. If blank, all active locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

WebOnly: Select to only include items that will be exported for Ecommerce purposes.

SkipIfZero: Select to only include items with non-zero stock or order quantities.

The result received is as follows:

{"Count":1,"RecNo":88,"ErrorInfo":"","Stock":[{StockQtyHeaderRecord,"StockLoc":[{StockLocRecord}]}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

GetStockQuantitiesFor (ConnectID: String; ItemList: TJSONArray; LocNo: String)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetStockQuantitiesFor"/ConnectID/
With {"_parameters":[ConnectID,[{"ItemNo":"Item 1"},{"ItemNo":"Item 2"}..],LocNo]} **in the body.**

or

GetStockQuantitiesForOpt2 (ConnectID: String; LocNo: String; ForItems: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetStockQuantitiesForOpt2"/ConnectID/LocNo/
With {"ItemList":[{"ItemNo":"Item 1"},{"ItemNo":"Item 2"}..]} **in the body.**

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

ItemList is a JSON Array of items for which the quantities are required. The format is as follows:

[{"ItemNo":"Item 1"},{"ItemNo":"Item 2"}]

LocNo represents the required stock location. If blank, all active locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

The result received is the same as for *GetStockQuantities* and is as follows:

{"Count":1,"RecNo":88,"ErrorInfo":"","Stock":[{StockQtyHeaderRecord,"StockLoc":[{StockLocRecord}]}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

Note that only valid item numbers will be included in the result.

StockHeaderRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Description	C	40	0			Item Description
ItemType	C	1	0			Item Type

GetStockPictures (ConnectID, MinItemNo, MaxItemNo: String; WebOnly, SkipIfBlank: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetStockPictures"/ConnectID/MinItemNo/ MaxItemNo/WebOnly/SkipIfBlank/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

WebOnly: Select to only include items that will be exported for Ecommerce purposes.

SkipIfBlank: Select to only include items with pictures.

The result received is as follows:

{"Count":1,"RecNo":88,"ErrorInfo":"","Stock":[{StockHeaderRecord,"Picture":"..."}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The picture value is a base64 encoded string of the original JPG file.

GetDetailDescriptions (ConnectID, MinItemNo, MaxItemNo: String; WebOnly, SkipIfBlank: Boolean;

RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDetailDescriptions"/ConnectID/MinItemNo/ MaxItemNo/WebOnly/SkipIfBlank/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

WebOnly: Select to only include items that will be exported for Ecommerce purposes.

SkipIfBlank: Select to only include items with detail descriptions.

The result received is as follows:

{"Count":1,"RecNo":88,"ErrorInfo":"","Stock":[{StockHeaderRecord,"DetailDescription":"..."}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

function CreateStockItem (ConnectID: String; StockRecord: TJSONValue): TJSONValue; (POST)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateStockItem"/ConnectID/
With `{StockRecord}` **in the body.**

StockRecord is the StockRecord structure listed above. The ItemNo and Description values are mandatory. All the other values will be filled with the default values where applicable. ChangeDate and ChangeTime are read only fields. **If any of the Category, Brand, Group or ItemClass codes doesn't exist, it will be created with the description (and WebList setting in the case of Category) supplied.**

The result received is as follows:

{"ErrorInfo":"","StockInfo":{StockRecord}} if successful or

{"ErrorInfo":"Item number required or whatever the error may be."} if unsuccessful.

StockPriceRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
SellingPrice1	N	18	4			Selling Price 1 (Excl)
SellingPrice2	N	18	4			Selling Price 2 (Excl)
SellingPrice3	N	18	4			Selling Price 3 (Excl)
SellingPrice4	N	18	4			Selling Price 4 (Excl)
SellingPrice5	N	18	4			Selling Price 5 (Excl)
SellingPrice6	N	18	4			Selling Price 6 (Excl)

UpdateItemPrice (ConnectID: String; StockPriceRec: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"UpdateItemPrice"/ConnectID/
With `{StockPriceRecord}` **in the body.**

Prices must exclude tax and can only be updated if item's price structure is set to 0 (Fixed Prices).

StockPriceRec is the StockPriceRecord listed above.

The result received is as follows:

`{"ErrorInfo":"","StockInfo":{StockPriceRecord}}` if successful or

`{"ErrorInfo":"Whatever the error may be."}` if unsuccessful.

DetailDescriptionRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
DetailDescription	C	0	0			Detail Description

UpdateDetailDescription (ConnectID: String; DetailDescriptionRec: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"UpdateDetailDescription"/ConnectID/
With `{DetailDescriptionRecord}` **in the body.**

DetailDescriptionRec is the DetailDescriptionRecord listed above. A length of 0 means that it is a variable length field.

The result received is as follows:

`{"ErrorInfo":"","StockInfo":{DetailDescriptionRecord}}` if successful or

`{"ErrorInfo":"Whatever the error may be."}` if unsuccessful.

LocationRecord:

Key	Type	Len	Dec	Values	Default	Description
LocNo	C	6	0			Stock Location
Description	C	30	0			Location Description

GetLocations (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetLocations"/ConnectID/

The result received is as follows:

`{"ErrorInfo":"," "Locations": [{LocationRecord}]}` if successful or

`{"ErrorInfo":"Whatever the error may be."}` if unsuccessful.

PriceInfoRecord:

Key	Type	Len	Dec	Values	Default	Description
AccountDiscount	N	18	2			Account Discount Percentage
ItemDiscount	N	18	2			Item Discount Percentage
FromDate	D	8	0			Discount From Date
ToDate	D	8	0			Discount To Date
AccPriceList	C	1	0	Y/N	N	Price List Used
PriceAccNo	C	6	0			Account Number for Pricing
PriceType	N	6	0			Current Account Price Type
ItemHasPromo	C	1	0	Y/N	N	Promotional Price Active
SupRebate	C	1	0			Account Qualifies for Supplier Rebate
SupRebateValue	N	18	2			Supplier Rebate
ItemCost	N	18	4			Item Cost
ProPrice	N	18	4			Promotional Price
ProPriceType	C	1	0	Y/N	N	Qualifying Price Types (Array of 6)
ProFromDate	D	8	0			Promotion From Date
ProToDate	D	8	0			Promotion To Date
ProQuantity	N	18	2			Promotion Minimum Quantity
ProMaxQuantity	N	18	2			Promotion Maximum Quantity
RetailStandard	N	18	4			*Retail Price Excluding Discounts
RetailSuggested	N	18	4			*Retail Price Including Discounts
RetailAccount	N	18	4			*Account Retail Price
StockOnHand	N	18	2			**Stock on Hand
StockOnOrder	N	18	2			**Stock on Sales Orders
StockOnBackOrder	N	18	2			**Stock On Purchase Orders
StockAvaialble	N	18	2			**Stock Available
AccStockNo	C	25	0			Account Stock Number

***Array of 6 prices excluding tax. (Price Type 1 = first element, Price Type 2 = second element…)**
****Array of 41 quantities. (Location 0 = first element, Location 1 = second element…)**

GetAccountPrice (ConnectID, ItemNo, AccountNo: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetAccountPrice"/ConnectID/ItemNo/AccNo /

The result received is as follows:

{"ErrorInfo":"," "AccPriceInfo":{PriceInfoRecord}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

Notes on price determination:

1. If ItemHasPromo = 'Y': The item discount is ignored.
2. If AccPriceList = 'Y': All discounts (item, account and promotion) are ignored and all the price arrays are set to the price list value for all price types.
3. The supplier rebate adjusts the item's cost before variable prices are calculated. Add the rebate value to the returned item cost if the normal cost value is required.

4. Use the prices in RetailSuggested (includes promotional prices) and RetailAccount (excludes promotional prices) depending on the settings for promotional prices and the quantity involved. The prices in RetailStandard can be used to determine the discount allowed if required.

IMPORTANT: If SupRebate=Y is returned, Sales Orders and Quotations must be created with SupRebate=Y (Refer to SalesOrderRecord and QuotationRecord) as this indicates that the account qualified for supplier rebates at the time of creation.

Additional information like the stock quantities and the account's item number is returned for added convenience.

10: Serial Numbers

SerialNoRecord:

Key	Type	Len	Dec	Values	Default	Description
SerialNo	C	40	0			Serial Number
ItemNo	C	25	0			Item Number
InDocType	C	1	0			In Document Type
InDocNo	C	8	0			In Document Number
InDocDate	D	8	0			In Document Date
OutDocType	C	1	0			Out Document Type
OutDocNo	C	8	0			Out Document Number
OutDocDate	D	8	0			Out Document Date
OrderNo	C	8	0			Order Number
OrderType	C	1	0			Order Type
GRVNo	C	8	0			GRV Number
SupAccNo	C	6	0			Supplier Account Number
GRVDate	D	8	0			GRV Date
SwopSerialNo	C	40	0			Swop Serial Number
StockType	C	1	0			Stock Type
Parent	C	40	0			Parent Serial Number
Location	C	6	0			Stock Location
Group	C	12	0			Serial Number Assembly Group

GetSerialNo (ConnectID, SerialNo: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetSerialNo”/ConnectID/SerialNo/

The result received is as follows:

{"ErrorInfo":"","SerialNumberInfo":{SerialNoRecord}} if successful or

{"ErrorInfo":"Serial number not found or whatever the error may be."} if unsuccessful.

DocSerialNoRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
SerialNo	C	40	0			Serial Number
StockType	C	1	0			Stock Type
Parent	C	40	0			Parent Serial Number
Location	C	6	0			Stock Location
Group	C	12	0			Serial Number Assembly Group

Stock Types:

N – Normal stock
F – Faulty stock
P – Supplier stock
S – Swop-out stock
T – Technician stock

C – Sold stock
U – Support stock
R – Returned stock
B – Branch stock
O – Consignment stock
W – WIP stock
L – Build-up stock
Z – Written off/Replaced stock

GetDocSerialNo (ConnectID: String; DocType: Integer; DocNo: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDocSerialNo"/DocType/DocNo/

DocType: The following document types are allowed:

1. Goods Received Voucher (Entries: GMTUgo)
2. Debit Note (Entries: DEXVhp)
3. Sales Order
4. Faulty Stock Send (Entry: F)
5. Faulty Stock Received (Entry: R)
6. Credit Note (Entry: C)
7. Invoice (Entry: I)
8. Stock Transfer
9. Customer Repair Order
10. Branch Order
11. Production Order

DocNo: The required document number. The document number will automatically be left padded with zeros up to the required length.

The result received is as follows:

{"ErrorInfo":"","SerialNumberInfo":[{DocSerialNoRecord}]} if successful or

{"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

SerialNoWarrantyRecord:

Key	Type	Len	Dec	Values	Default	Description
SerialNo	C	40	0			Serial Number
ItemNo	C	25	0			Item Number
Description	C	40	0			Item Description
Brand	C	15	0			Brand Code
BrandDescription	C	30	0			Brand Description
Warranty	I	6	0			Item Warranty Months
InvNo	C	8	0			Invoice Number
InvDate	D	8	0			Invoice Date
AccNo	C	6	0			Customer Account Number
Name	C	40	0			Customer Name

GetSerialNoWarranty (ConnectID, SerialNo: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetSerialNoWarranty"/ConnectID/SerialNo/

The result received is as follows:

{"ErrorInfo":"","SerialNumberInfo":{SerialNoWarrantyRecord *,”SerialNumberAssemblyInfo”: [SerialNoWarrantyRecord**]}} if successful or

{"ErrorInfo":"Serial number not found or whatever the error may be."} if unsuccessful.

*: The parent serial number’s information.

**: If the serial number that was queried is a parent serial number or if it is part of a sub-assembly, SerialNumberAssemblyInfo will contain an array of all the serial numbers in the sub-assembly. If not, the array will be empty.

11: Quotations

QuotationRecord:

Key	Type	Len	Dec	Values	Default	Description
QuoteNo	C	8	0			Quotation Number
AccNo	C	6	0			Account Number
LocNo	C	6	0			Stock location
QuoteDate	D	8	0			Quotation Date
TotalExcl	N	18	2			Total Excl
TotalTax	N	18	2			Total Tax
CurrencyCode	C	2	0			Currency Code
ExchangeRate	N	18	7			Quotation Exchange Rate
CustomerRef	C	20	0			Customer Reference
DebName	C	40	0			Customer Name
Addr1	C	40	0			Address Line 1
Addr2	C	40	0			Address Line 2
Addr3	C	40	0			Address Line 3
PCode	C	8	0			Postal Code
DelName	C	40	0			Delivery Name
DelAddr1	C	40	0			Delivery Address Line 1
DelAddr2	C	40	0			Delivery Address Line 2
DelAddr3	C	40	0			Delivery Address Line 3
DelAddr4	C	40	0			Delivery Address Line 4
DelPCode	C	8	0			Delivery Postal Code
DelInstruc1	C	40	0			Delivery Instructions Line 1
DelInstruc2	C	40	0			Delivery Instructions Line 2
DelInstruc3	C	40	0			Delivery Instructions Line 3
DelInstruc4	C	40	0			Delivery Instructions Line 4
DelInstruc5	C	40	0			Delivery Instructions Line 5
DelInstruc6	C	40	0			Delivery Instructions Line 6
Message1	C	40	0			Message Line 1
Message2	C	40	0			Message Line 2
Message3	C	40	0			Message Line 3
RepCode	C	3	0			Sales Rep Code
TaxNo	C	30	0			Tax Registration Number
Status	C	0	0			Quotation Status
QuoteType	C	1	0			Quotation Type**
SupRebate	C	1	0	Y/N	N	Supplier Rebate

** I = Proforma Invoice, Q = Quotation

QuotationDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Group	N	2				Group Number
Quantity	N	18	2			Item Quantity
LineTotalExcl	N	18	2			Line Total Excl
TaxCode	I	8	0			Tax Code
LineTotalTax	N	18	2			Line Total Tax
Description	C	40	0			Item Description

UnitCost	N	18	4	Unit Cost

GroupDesrciptionRecord:

Key	Type	Len	Dec	Values	Default	Description
Group	N	2				Group Number
Description	C	40				Group Description
Quantity	N	18	2			Group Quantity

GetQuotationsByDate (ConnectID, MinDate, MaxDate, LocNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuotationsByDate"/ConnectID/MinDate /MaxDate/LocNo/RecNo/Count/

GetQuotationsByRep (ConnectID, MinRep, MaxRep, LocNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuotationsByRep"/ConnectID/MinRep /MaxRep/LocNo/RecNo/Count/

GetQuotationsByName (ConnectID, MinName, MaxName, LocNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuotationsByNAme"/ConnectID/MinName /MaxName/LocNo/RecNo/Count/

GetQuotationsByAccNo (ConnectID, MinAccNo, MaxAccNo, LocNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuotationsByAccNo"/ConnectID/MinAccNo /MaxAccNo/LocNo/RecNo/Count/

GetQuotationsByQuoteNo (ConnectID, MinQuoteNo, MaxQuoteNo, LocNo: String; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuotationsByQuoteNo"/ConnectID/MinQuoteNo /MaxQuoteNo/LocNo/RecNo/Count/

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","Quotations":[{QuotationRecord, "QuotationDetail":[{QuotationDetailRecord}], "GroupDetail":[{GroupDescriptionRecord}]}]} if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of QuotationRecord structures that includes an array of QuotationDetailRecord structures (Key: QuotationDetail) on each transaction record.

The total excluding and tax amounts are in the debtor account's currency. Use the ExchangeRate value to calculate the local currency amount if required.

CreateQuotation (ConnectID: String; QuoteRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateQuotation"/ConnectID/
With
`{QuotationRecord,"QuotationDetail":[{QuotationDetailRecord}],"GroupDetail":[{GroupDescriptionRecord}]}` **in the body.**

QuoteRecord is a combination of the QuotationRecord, QuotationDetailRecord and GroupDescriptionRecord structures listed above:

{QuotationRecord, "QuotationDetail":[{QuotationDetailRecord}],"GroupDetail":[{GroupDescriptionRecord}]}
QuoteNo, Status and UnitCost are read-only fields. QuoteDate will always be the current date. All the other values will be filled with the default values or information retrieved from the debtor account where applicable. **Only local currency debtor accounts are allowed. The maximum number of detail lines is currently 1000.**

The result received is as follows:

{"ErrorInfo":"","QuotationInfo":{QuotationRecord,"QuotationDetail":[{QuotationDetailRecord}], "GroupDetail":[{GroupDescriptionlRecord}]}} if successful or

{"ErrorInfo":"Invalid account number or whatever the error may be."} if unsuccessful.

QuotationExpiredResultRecord:

Key	Type	Len	Dec	Values	Default	Description
QuoteNo	C	8	0			CRO Number
Changed	L	1	0			Status Changed
Status	C	0	0			Item Status

SetQuotationExpired (ConnectID: String; QuoteNo: String; Expired: Boolean)

POST/GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"SetQuotationExpired"/ConnectID/Expired/

Set **Expired** to true to change the quotation's status to expired or to false to set the quotation's status to active.

The result received is as follows:

{"ErrorInfo":"","QuotationInfo":{QuotationExpiredResultRecord}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

12: Sales Orders

SalesOrderRecord:

Key	Type	Len	Dec	Values	Default	Description
OrderNo	C	8	0			Sales Order Number
AccNo	C	6	0			Account Number
LocNo	C	6	0			Stock location
QuoteNo	C	8	0			Quotation Number
OrderType	C	1	0	N	N	Order Type
OrderDate	D	8	0			Order Date
DateRequired	D	8	0			Date Required
ExpiryDate	D	8	0			Expiry Date
TotalExcl	N	18	2			Total Excl
TotalTax	N	18	2			Total Tax
CurrencyCode	C	2	0			Currency Code
ExchangeRate	N	18	7			Sales Order Exchange Rate
CustomerRef	C	20	0			Customer Reference
DebName	C	40	0			Customer Name
Addr1	C	40	0			Address Line 1
Addr2	C	40	0			Address Line 2
Addr3	C	40	0			Address Line 3
PCode	C	8	0			Postal Code
DelName	C	40	0			Delivery Name
DelAddr1	C	40	0			Delivery Address Line 1
DelAddr2	C	40	0			Delivery Address Line 2
DelAddr3	C	40	0			Delivery Address Line 3
DelAddr4	C	40	0			Delivery Address Line 4
DelPCode	C	8	0			Delivery Postal Code
DelInstruc1	C	40	0			Delivery Instructions Line 1
DelInstruc2	C	40	0			Delivery Instructions Line 2
DelInstruc3	C	40	0			Delivery Instructions Line 3
DelInstruc4	C	40	0			Delivery Instructions Line 4
DelInstruc5	C	40	0			Delivery Instructions Line 5
DelInstruc6	C	40	0			Delivery Instructions Line 6
DeliveryMethod	C	1	0	C/D/R	C	Delivery Method
RepCode	C	3	0			Sales Rep Code
TaxNo	C	30	0			Tax Registration Number
Approved	C	1	0	Y/N	N	Order Approved
InvLayoutName	C	20	0			Invoice Layout Name
NumberOfItems	N	18	2			Number Of Items
InvoiceNumbers	C	0	0			Invoice Numbers
Status	C	0	0			Sales Order Status
Assembly	C	1	0	Y/N	N	Contains Assembly Item/s
SupRebate	C	1		Y/N	N	Supplier Rebate

SalesOrderDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
Entry	C	1	0			Entry Type

DocNo	C	8	0			Document Number
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Item Quantity
LineTotalExcl	N	18	2			Line Total Excl
TaxCode	I	6	0			Tax Code
LineTotalTax	N	18	2			Line Total Tax
Description	C	40	0			Item Description
UnitCost	N	18	4			Unit Cost

GroupDesrciptionRecord:

Key	Type	Len	Dec	Values	Default	Description
Group	N	2				Group Number
Description	C	40				Group Description
Quantity	N	18	2			Group Quantity

GetSalesOrdersByDate (ConnectID, MinDate, MaxDate, LocNo: String; OutstandingOnly, ListTransactions: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetSalesOrdersByDate"/ConnectID/MinDate /MaxDate/LocNo/OutstandingOnly/ListTransactions/RecNo/Count/

GetSalesOrdersByRep (ConnectID, MinRep, MaxRep, LocNo: String; OutstandingOnly, ListTransactions: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetSalesOrdersByRep"/ConnectID/MinRep /MaxRep/LocNo/OutstandingOnly/ListTransactions/RecNo/Count/

GetSalesOrdersByAccNo (ConnectID, MinAccNo, MaxAccNo, LocNo: String; OutstandingOnly, ListTransactions: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetSalesOrdersByAccNo"/ConnectID/MinAccNo /MaxAccNo/LocNo/OutstandingOnly/ListTransactions/RecNo/Count/

GetSalesOrdersByOrderNo (ConnectID, MinOrderNo, MaxOrderNo, LocNo: String; OutstandingOnly, ListTransactions: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetSalesOrdersByOrderNo"/ConnectID/MinOrderNo /MaxOrderNo/LocNo/OutstandingOnly/ListTransactions/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

To list the last n records, set **RecNo** to -1 and **Count** to the number of records required. (The record number of the last record visited will not be returned meaning that calling the function again, will **NOT** continue from where it stopped. **Count** must therefore be high enough to include all the required records.) Setting **RecNo** to -1, will cause FinconRESTServer to traverse the data in reverse order. There is no need to swop the minimum and maximum values as the normal rules still apply. In reverse order, traversing will start at the last record for MaxValue (or the last record in the table if MaxValue is blank) and list up to the first record for MinValue or the first record in the table or until **Count** has been reached, whichever comes first.

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

OutstandingOnly: Select to only include outstanding orders.

ListTransactions: Select to include detail transactions – this will include invoice transactions.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","SalesOrders":[{SalesOrderRecord, “SalesOrderDetail":[{SalesOrderDetailRecord}],"GroupDetail":[{GroupDescriptionRecord}]}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of SalesOrderRecord structures that includes an array of SalesOrderDetailRecord structures (Key: SalesOrderDetail) on each transaction record.

GetCancelledSalesOrdersByDate (ConnectID, MinDate, MaxDate, LocNo: String; Boolean; RecNo, Count: Integer)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetCancelledSalesOrdersByDate”/ConnectID/MinDate /MaxDate/LocNo/RecNo/Count/

GetCancelledSalesOrdersByRep (ConnectID, MinRep, MaxRep, LocNo: String; RecNo, Count: Integer)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetCancelledSalesOrdersByRep”/ConnectID/MinRep /MaxRep/LocNo/ RecNo/Count/

GetCancelledSalesOrdersByAccNo (ConnectID, MinAccNo, MaxAccNo, LocNo: String; RecNo, Count: Integer)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetCancelledSalesOrdersByAccNo”/ConnectID /MinAccNo/MaxAccNo/LocNo/ RecNo/Count/

GetCancelledSalesOrdersByOrderNo (ConnectID, MinOrderNo, MaxOrderNo, LocNo: String; RecNo, Count: Integer)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetCancelledSalesOrdersByOrderNo”/ConnectID /MinOrderNo/MaxOrderNo/LocNo /RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to ‘0’. Or set LocNo to ‘0,1,10’ to include locations 00, 01 and 10.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","SalesOrders":[{SalesOrderRecord, “SalesOrderDetail":[{SalesOrderDetailRecord}]}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of SalesOrderRecord structures that includes an array of SalesOrderDetailRecord structures (Key: SalesOrderDetail) on each transaction record.

The total excluding and tax amounts are in the debtor account’s currency. Use the ExchangeRate value to calculate the local currency amount if required.

GetGiftCardInfo (ConnectID, GiftCardNo: String)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetGiftCardInfo”/ConnectID/GiftCardNo/

The result received is as follows:

{"ErrorInfo":","Balance”:123.45}
if successful or

{"ErrorInfo":"No records found or whatever the error may be." ,"Balance”:0} if unsuccessful.

SalesOrderPaymentRecord:

Key	Type	Len	Dec	Values	Default	Description
ReceiptNo	C	8	0			Receipt Number
PayType	C	1	0			Payment Type
CardNo	C	20	0			Card Number
Amount	N	18	2			Payment Amount

SalesOrderGiftCardRecord:

Key	Type	Len	Dec	Values	Default	Description
ReceiptNo	C	8	0			Receipt Number
GiftCardNo	C	20	0			Gift Card Number
Comment	C	40	0			Any Applicable Comment
Amount	N	18	2			Payment Amount
Balance	N	18	2			Remaining Balance

CreateSalesOrder (ConnectID: String; SetAsApproved: Boolean; OrderRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/”CreateSalesOrder”/ConnectID/SetAsApproved/
With {SalesOrderRecord,“SalesOrderDetail”:[{SalesOrderDetailRecord}],“GroupDetail”:[{GroupDescriptionRecord}], “SalesOrderPayment”:{SalesOrderPaymentRecord},“SalesOrderGiftCard”:{SalesOrderGiftCardRecord}} **in the body.**

OrderRecord is a combination of the SalesOrderRecord, SalesOrderDetailRecord, GroupDescriptionRecord (Optional), SalesOrderPayment (Optional) and SalesOrderGiftCard (Optional) structures listed above:

{SalesOrderRecord, "SalesOrderDetail":[{SalesOrderDetailRecord}],"GroupDetail":[{GroupDescriptionRecord}], "SalesOrderPayment":{SalesOrderPaymentRecord},"SalesOrderGiftCard":{SalesOrderGiftCardRecord}}

OrderNo, Entry, DocNo, Status, UnitCost and ReceiptNo are read-only fields. OrderType will always be ‘N’, OrderDate will always be the current date. Approved will be set to ‘Y’ if the **SetAsApproved** parameter is true and to ‘N’ if **SetAsApproved** is false. All the other values will be filled with the default values or information retrieved from the debtor account where applicable. **Only local currency debtor accounts are allowed. The maximum number of detail lines is 1000.**

PayType must be ‘C’ for card payments and ‘T’ for transfers. **CardNo** must be supplied for card payments.

The result received is as follows:

{"ErrorInfo":"","SalesOrderInfo":{SalesOrderRecord,"SalesOrderDetail":[{SalesOrderDetailRecord}], "GroupDetail":[{GroupDescriptionRecord}],"SalesOrderPayment":SalesOrderPaymentRecord}, "SalesOrderGiftCard":SalesOrderGiftCardRecord}}} if successful or

{"ErrorInfo":"Invalid account number or whatever the error may be."} if unsuccessful.

CreateSalesOrderPayment (ConnectID, SalesOrderNo: String; SetAsApproved: Boolean; PaymentRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/”CreateSalesOrderPayment”/ConnectID/SalesOrderNo /SetAsApproved/
With {“SalesOrderPayment”:{SalesOrderPaymentRecord},“SalesOrderGiftCard”:{SalesOrderGiftCardRecord}} **in the body.**

PaymentRecord is a combination of the SalesOrderPayment and/or the SalesOrderGiftCard structures listed above.

The result received is as follows:

{"ErrorInfo":"","OrderNo":"1234578","SalesOrderPayment":SalesOrderPaymentRecord}, SalesOrderGiftCard:{SalesOrderGiftCardRecord}} if successful or

{"ErrorInfo":"Invalid order number or whatever the error may be."} if unsuccessful.

SalesOrderCancelResultRecord:

Key	Type	Len	Dec	Values	Default	Description
OrderNo	C	8	0			Order Number
Changed	L	1	0			Status Changed
Status	C	10	0			New Status
HadSerial	L	1	0			Had Allocated Serial Numbers
HadPicked	L	1	0			Had Picked Quantities

CancelSalesOrder (ConnectID: String; SalesOrderNo: String)

POST/GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"CancelSalesOrder"/ConnectID/SalesOrderNo/

SalesOrderNo: The sales order number.

Serial numbers and picked quantities will be unallocated depending on the rights granted otherwise an error message regarding allocated serial numbers and picked quantities will be received.

The result received is as follows:

{"ErrorInfo":"","SalesOrderInfo":{SalesOrderCancelResultRecord}} if successful or

{"ErrorInfo":"Invalid order number whatever the error may be."} if unsuccessful.

SalesOrderPickedRecord:

Key	Type	Len	Dec	Values	Default	Description
OrderNo	C	8	0			Order Number
AccNo	C	6	0			Account Number
DebName	C	40	0			Customer Name
LocNo	C	6	0			Stock location
OrderType	C	1	0	N	N	Order Type
OrderDate	D	8	0			Order Date
DateRequired	D	8	0			Date Required
RepCode	C	3	0			Sales Rep Code

SalesOrderPickedDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Item Quantity
PickedQuantity	N	18	2			Picked Quantity
SerialNo	C	1	0			Serial Number Required

GetPickedOrdersByDate (ConnectID, MinDate, MaxDate, LocNo: String; IncludeUnpicked, IncludeSerialNoItems: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetPickedOrdersByDate"/ConnectID/MinDate /MaxDate/IncludeUnpicked/IncludeSerialNoItems/RecNo/Count/

GetPickedOrdersByRep (ConnectID, MinRep, MaxRep, LocNo: String; IncludeUnpicked, IncludeSerialNoItems: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetPickedOrdersByRep"/ConnectID/MinRep /MaxRep/IncludeUnpicked/IncludeSerialNoItems/RecNo/Count/

GetPickedOrdersByAccNo (ConnectID, MinAccNo, MaxAccNo, LocNo: String; IncludeUnpicked, IncludeSerialNoItems: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetPickedOrdersByAccNo"/ConnectID/MinAccNo /MaxAccNo/IncludeUnpicked/IncludeSerialNoItems/RecNo/Count/

GetPickedOrdersByOrderNo (ConnectID, MinOrderNo, MaxOrderNo, LocNo: String; IncludeUnpicked, IncludeSerialNoItems: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetPickedOrdersByOrderNo"/ConnectID/MinOrderNo /MaxOrderNo/IncludeUnpicked/IncludeSerialNoItems/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

IncludeUnpicked: Select to only include unpicked items.

IncludeSerialNoItems: Select to include items requiring serial numbers.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","SalesOrders":[{SalesOrderPickedRecord, SalesOrderDetail":[{SalesOrderPickedDetailRecord}]}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of SalesOrderPickedRecord structures that includes an array of SalesOrderPickedDetailRecord structures (Key: SalesOrderDetail) on each transaction record.

SalesOrderPickResultRecord:

Key	Type	Len	Dec	Values	Default	Description
OrderNo	C	8	0			Order Number
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Item Quantity
PickedQuantity	N	18	2			Picked Quantity
MarkComplete	L	0	0			Mark Order Complete

PickSalesOrder (ConnectID, SalesOrderNo, ItemNo: String; Picked: Double; MarkComplete: Boolean)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"PickSalesOrder"/ConnectID/SalesOrderNo /ItemNo/Picked/MarkComplete/

SalesOrderNo: The sales order number.

ItemNo: The item number of the item to be picked.

Picked: The picked quantity.

MarkComplete: Mark the order as completed (Ready to invoice).

The result received is as follows:

{"ErrorInfo":"","SalesOrderInfo":{SalesOrderPickResultRecord}} if successful or

{"ErrorInfo":"The picked quantity cannot be more than 10.00 or whatever the error may be."} if unsuccessful.

13: Purchase Orders

PurchaseOrderRecord:

Key	Type	Len	Dec	Values	Default	Description
OrderNo	C	8	0			Purchase Order Number
AccNo	C	6	0			Account Number
LocNo	C	6	0			Stock location
OrderDate	D	8	0			Order Date
DeliveryDate	D	8	0			Delivery Date
ShippingDate	D	8	0			Shipping Date
TotalExcl	N	18	2			Total Excl
TotalTax	N	18	2			Total Tax
CurrencyCode	C	2	0			Currency Code
ExchangeRate	N	18	7			Purchase Order Exchange Rate
SupplierRef	C	20	0			Supplier Reference
CredName	C	40	0			Supplier Name
Addr1	C	40	0			Address Line 1
Addr2	C	40	0			Address Line 2
Addr3	C	40	0			Address Line 3
PCode	C	8	0			Postal Code
CollName	C	40	0			Collection Name
CollAddr1	C	40	0			Collection Address Line 1
CollAddr2	C	40	0			Collection Address Line 2
CollAddr3	C	40	0			Collection Address Line 3
CollAddr4	C	40	0			Collection Address Line 4
CollPCode	C	8	0			Collection Postal Code
Message1	C	40	0			Message Line 1
Message2	C	40	0			Message Line 2
Message3	C	40	0			Message Line 3
Comment	C	80	0			Order Comment
Cheque	C	6	0			Cheque
Approved	C	1	0	Y/N	N	Order Approved
NumberOfItems	N	18	2			Number Of Items
GRVNumbers	C	0	0			GRV Numbers
Status	C	0	0			Order Status

PurchaseOrderDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
Entry	C	1	0			Entry Type
DocNo	C	8	0			Document Number
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Item Quantity
LineTotalExcl	N	18	2			Line Total Excl
TaxCode	I	6	0			Tax Code
LineTotalTax	N	18	2			Line Total Tax
Description	C	40	0			Item Description
SalesOrderNo	C	8	0			Sales Order Number

GetPurchaseOrdersByDate (ConnectID, MinDate, MaxDate, LocNo: String; OutstandingOnly, ListTransactions: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetPurchaseOrdersByDate"/ConnectID/MinDate /MaxDate/LocNo/OutstandingOnly/ListTransactions/RecNo/Count/

GetPurchaseOrdersByAccNo (ConnectID, MinAccNo, MaxAccNo, LocNo: String; OutstandingOnly, ListTransactions: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetPurchaseOrdersByAccNo"/ConnectID/MinAccNo /MaxAccNo/LocNo/OutstandingOnly/ListTransactions/RecNo/Count/

GetPurchaseOrdersByOrderNo (ConnectID, MinOrderNo, MaxOrderNo, LocNo: String; OutstandingOnly, ListTransactions: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetPurchaseOrdersByOrderNo"/ConnectID /MinOrderNo/MaxOrderNo/LocNo/OutstandingOnly/ListTransactions/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

OutstandingOnly: Select to only include outstanding orders.

ListTransactions: Select to include detail transactions – this will include goods received voucher transactions.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","PuchaseOrders":[{PurchaseOrderRecord, PurchaseOrderDetail":[{PurchaseOrderDetailRecord}]}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of PurchaseOrderRecord structures that includes an array of PurchaseOrderDetailRecord structures (Key: PurchaseOrderDetail) on each transaction record.

The total excluding and tax amounts are in the creditor account's currency. Use the ExchangeRate value to calculate the local currency amount if required.

CreatePurchaseOrder (ConnectID: String; SetAsApproved: Boolean; OrderRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreatePurchaseOrder"/ConnectID/SetAsApproved/ With `{PurchaseOrderRecord,"PurchaseOrderDetail":[{PurchaseOrderDetailRecord}]}` **in the body.**

OrderRecord is a combination of the PurchaseOrderRecord and PurchaseOrderDetailRecord structures listed above:

{PurchaseOrderRecord, "PurchaseOrderDetail":[{PurchaseOrderDetailRecord}]}

OrderNo, Entry, DocNo and Status are read-only fields. OrderType will always be 'P', OrderDate will always be the current date. Approved will be set to 'Y' if the **SetAsApproved** parameter is true and to 'N' if **SetAsApproved** is false. All the other values will be filled with the default values or information retrieved from the creditor account where applicable. **Only local currency and non-import creditor accounts are allowed. The maximum number of detail lines is 1000.**

The result received is as follows:

{"ErrorInfo":"","PurchaseOrderInfo":{PurchaseOrderRecord,"PurchaseOrderDetail":[{PurchaseOrderDetailRecord}]}}
if successful or

{"ErrorInfo":"Invalid account number or whatever the error may be."} if unsuccessful.

14: Production Orders

ProductionOrderRecord:

Key	Type	Len	Dec	Values	Default	Description
OrderNo	C	8	0			Production Order Number
LocNo	C	6	0			Stock location
OrderDate	D	8	0			Order Date
ProductionDate	D	8	0			Production Date
Remark	C	40	0			Order Remark
Status	C	0	0			Order Status

ProductionOrderDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
DocNo	C	8	0			Document Number
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Item Quantity
Description	C	40	0			Item Description
SalesOrderNo	C	8	0			Sales Order Number

GetProductionOrdersByDate (ConnectID, MinDate, MaxDate, LocNo: String; OutstandingOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetProductionOrdersByDate"/ConnectID/MinDate /MaxDate/LocNo/OutstandingOnly/RecNo/Count/

GetProductionOrdersByProdDate (ConnectID, MinDate, MaxDate, LocNo: String; OutstandingOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetProductionOrdersByProdDate"/ConnectID/MinDate /MaxDate/LocNo/OutstandingOnly/RecNo/Count/

GetProductionOrdersByOrderNo (ConnectID, MinOrderNo, MaxOrderNo, LocNo: String; OutstandingOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetProductionOrdersByOrderNo"/ConnectID /MinOrderNo/MaxOrderNo/LocNo/OutstandingOnly/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

LocNo represents the required stock location. If blank, all locations are included. Should you require just the default location, set LocNo to '0'. Or set LocNo to '0,1,10' to include locations 00, 01 and 10.

OutstandingOnly: Select to only include outstanding orders.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","ProductionOrders":[{ProductionOrderRecord, ProductionOrderDetail":[{ProductionOrderDetailRecord}]}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of ProductionOrderRecord structures that includes an array of ProductionOrderDetailRecord structures (Key: ProductionOrderDetail) on each transaction record.

15: Stock Transfers

TransferRecord:

Key	Type	Len	Dec	Values	Default	Description
DocNo	C	8	0			Document Number
FromLocation	C	6	0			From Stock Location
ToLocation	C	6	0			To Stock Location
RefDoc	C	8	0			Reference Document Number
Remark	C	60	0			Transfer Remark

TransferDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Item Quantity
UnitOfMeasure	C	2	0			Unit Of Measure
Description	C	40	0			Item Description
UnitCost	N	18	4			Unit Cost
BinLocation	C	25	0			Bin Location

CreateStockTransfer (ConnectID: String; TransRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateStockTransfer"/ConnectID/
With `{TransferRecord,"TransferDetail":[{TransferDetailRecord}]}` **in the body.**

TransRecord is a combination of the TransferRecord and the TransferDetailRecord structures listed above:

```
{TransferRecord, TransferDetail":[{TransferDetailRecord}]}
```

DocNo, UnitOfMeasure, Description, UnitCost and BinLocation are read-only fields. **Items with serial number tracking enabled cannot be transferred. The maximum number of detail lines is currently 1000.**

The result received is as follows:

`{"ErrorInfo":"","TransferInfo":{TransferRecord,"TransferDetail":[{TransferDetailRecord}]}}` if successful or

`{"ErrorInfo":"Whatever the error may be."}` if unsuccessful.

16: Customer Repair Orders

CRORecord:

Key	Type	Len	Dec	Values	Default	Description
CRONo	C	8	0			CRO Number
TechCode	C	3	0			Technician Code
Assistant1	C	3	0			Assistant Technician Code 1
Assistant2	C	3	0			Assistant Technician Code 2
Assistant3	C	3	0			Assistant Technician Code 3
WSLocNo	C	6	0			Workshop Location Number
WSLocDescrip	C	20	0			Workshop Location Description
AccNo	C	6	0			Account Number
DebName	C	40	0			Customer Name
Addr1	C	40	0			Address Line 1
Addr2	C	40	0			Address Line 2
Addr3	C	40	0			Address Line 3
DelName	C	40	0			Delivery Name
DelAddr1	C	40	0			Delivery Address Line 1
DelAddr2	C	40	0			Delivery Address Line 2
DelAddr3	C	40	0			Delivery Address Line 3
DelAddr4	C	40	0			Delivery Address Line 4
DelPCode	C	8	0			Delivery Postal Code
Contact	C	30	0			Contact Person
TelNo	C	25	0			Telephone Number
FaxNo	C	25	0			Fax Number
EMail	C	100	0			Email Address
CustomerRef	C	20	0			Customer Reference
TaxNo	C	30	0			Tax Registration Number
RegNo	C	30	0			Company Registration Number
RegName	C	40	0			Registered Company Name
AreaCode	C	3	0			Area Code
AreaDescription	C	30	0			Area Description
Comment	C	40	0			CRO Comment
WayBillIn	C	30	0			Way Bill (Inbound)
CourierIn	C	30	0			Courier (Inbound)
CollDateIn	D	8	0			Collection Date (Inbound)
CollTimeIn	C	5	0			Collection Time (Inbound) (HH:MM)
ReturnRefIn	C	30	0			Return Reference (Inbound)
DateIn	D	8	0			Date Delivered (Inbound)
ChargeIn	N	18	2			Charge (Inbound)
WayBillOut	C	30	0			Way Bill (Outbound)
CourierOut	C	30	0			Courier (Outbound)
CollDateOut	D	8	0			Collection Date (Outbound)
CollTimeOut	C	5	0			Collection Time (Outbound) (HH:MM)
ReturnRefOut	C	30	0			Return Reference (Outbound)
DateOut	D	8	0			Date Delivered (Outbound)
ChargeOut	N	18	2			Charge (Outbound)
QuoteNo	C	8	0			Quotation Number
QuoteStatus	C	1	0	N/A/R		Quotation Status

QuoteReplyDate	D	8	0			Customer Reply Date
QuotePayDate	D	8	0			Customer Payment Date
QuotePayAmount	N	18	2			Customer Payment Amount
QuotePayRef	C	30	0			Customer Payment Reference
CRODate	D	8	0			CRO Date
DelDate	D	8	0			Delivery Date
CompletionDate	D	8	0			Date Completed
FetchedDate	D	8	0			Date Fetched
RepeatCall	C	1	0	Y/N	N	Repeat Call
AdditionalNotes	C	180	0			Additional Notes
PushID	C	20	0			CRO PushID
Status	C	0	0			CRO Status
Priority	C	1	0	1-5	1	Priority - 1 Highest, 5 Lowest
ExtReference	C	25	0			External reference

CROItemRecord:

Key	Type	Len	Dec	Values	Default	Description
SerialNo	C	40	0			Serial Number
ItemNo	C	25	0			Item Number
Description	C	40	0			Item Description
Notes	C	300	0			Item Notes
Fault	C	300	0			Item Fault
Diagnosis	C	300	0			Item Diagnosis
WorkDone	C	300	0			Item Work Done
Rejected	C	300	0			Repair Rejected Reason
WarrantyRepair	C	1	0	Y/N	N	Warranty Repair
ServiceType	C	1	0	R/I/P	R	Service Type
ContractNo	C	10	0			Contract Number
SwopSerialNo	C	40	0			Swop Serial Number
Status	C	0	0			Item Status
SubAccNo	C	6	0			Sub Account Number

CreateCRO (ConnectID: String; CRORecord, CROItemRecord: TJSONObject; AllowAll: Boolean)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateCRO"/ConnectID/
With `{"_parameters":[ConnectID,{CRORecord},{CROItemRecord},AllowAll]}` **in the body.**

or

CreateCROOpt2 (ConnectID: String; AllowAll: Boolean; CRORecord, CROItemRecord: TJSONObject)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateCROOpt2/ConnectID/AllowAll/
With `{CRORecord,"RepairOrderItem":{CROItemRecord}}` **in the body.**

AllowAll: Set to true to allow serial numbers not present in Fincon. (Rights dependent)

The minimum information required to generate a customer repair order, is the technician code (TechCode) in CRORecord and the serial number (SerialNo) in CROItemRecord. The rest of the fields will be populated where possible and returned. The Status will always be set to *Pending*.

The result received is as follows:

{"ErrorInfo":"","RepairOrderInfo":{CRORecord,"RepairOrderItem":{CROItemRecord}}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

CROVisitRecord:

Key	Type	Len	Dec	Values	Default	Description
RespondedDate	D	8	0			Date Responded
RespondedTime	C	5	0			Time Responded (HH:MM)
ArrivedDate	D	8	0			Date Arrived
ArrivedTime	C	5	0			Time Arrived (HH:MM)
CompletedDate	D	8	0			Date Completed
CompletedTime	C	5	0			Time Completed (HH:MM)
OdoMeterStart	N	18	0			ODO Meter Reading - Start
OdoMeterEnd	N	18	0			ODO Meter Reading - End
Meter1	N	18	2			Meter 1 Reading
Meter2	N	18	2			Meter 2 Reading

CROCompleteRecord:

Key	Type	Len	Dec	Values	Default	Description
Diagnosis	C	300	0			Item Diagnosis
WorkDone	C	300	0			Item Work Done
Rejected	C	300	0			Repair Rejected Reason
CompletionDate	D	8	0			Date Completed

CROCompleteResultRecord:

Key	Type	Len	Dec	Values	Default	Description
CRONo	C	8	0			CRO Number
SerialNo	C	40	0			Serial Number
Changed	L	1	0			Status Changed
Status	C	0	0			Item Status

CompleteCRO (ConnectID, CRONo, SerialNo: String)

POST/GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"CompleteCRO"/ConnectID/CRONo/SerialNo/

or

CompleteCROExt (ConnectID, CRONo, SerialNo: String; CompleteCRORecord: TJSONObject):

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CompleteCROExt"/ConnectID/CRONo/SerialNo/
With {CROCompleteRecord,"Visit1":{CROVisitRecord},"Visit2":{CROVisitRecord},"Visit3":{CROVisitRecord}, "Visit4":{CROVisitRecord},"Visit5":{CROVisitRecord}} **in the body.**

CompleteCRORecord is a combination of the CROCompleteRecord and 5 x CROVisitRecord (Optional) structures listed above:

{CroCompleteRecord, "Visit1":{CROVisitRecord},"Visit2":{CROVisitRecord}, "Visit3":{CROVisitRecord}, "Visit4":{CROVisitRecord},"Visit5":{CROVisitRecord}}

A CRO can be completed multiple times up until the CRO is approved, invoiced or closed. In the case of CompleteCROExt, the fields in CROCompleteRecord must always be populated. The visits are totally optional in the sense that you only need to include new or updated visit records.

CRONo: The customer repair order number.

SerialNo: The item's serial number.

The result received is as follows:

{"ErrorInfo":"","RepairOrderInfo":{CROCompleteResultRecord}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

CROLogRecord:

Key	Type	Len	Dec	Values	Default	Description
Date	D	8	0			Log Entry Date
Time	C	5	0			Log Entry Time
User	C	8	0			User
Action	C	250	0			Action

CRONotesRecord:

Key	Type	Len	Dec	Values	Default	Description
Date	D	8	0			Note Entry Date
Time	C	5	0			Note Entry Time
User	C	8	0			User
TechCode	C	3	0			Technician Code
Note	C	180	0			Note

CROStockRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Description	C	40	0			Description
UnitOfMeasure	C	2	0			Unit Of Measure
Quantity	N	18	2			Quantity

GetCROsByCRONo (ConnectID, MinCRONo, MaxCRONo, CROType: String; OutstandingOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCROsByCRONo"/ConnectID/MinCRONo /MaxCRONo/CROType/OutstandingOnly/RecNo/Count/

GetCROsByAccNo (ConnectID, MinAccNo, MaxAccNo, CROType: String; OutstandingOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCROsByAccNo"/ConnectID/MinAccNo /MaxAccNo/CROType/OutstandingOnly/RecNo/Count/

GetCROsByTechnician (ConnectID, MinTechCode, MaxTechCode, CROType: String; OutstandingOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCROsByTechnician"/ConnectID/MinTechCode /MaxTechCode/CROType/OutstandingOnly/RecNo/Count/

GetCROsByLocation (ConnectID, LocNo, CROType: String; OutstandingOnly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCROsByLocation"/ConnectID/LocNo /CROType/OutstandingOnly/RecNo/Count/

NOTE: Leaving LocNo blank, will only list the customer repair orders that are not assigned to a workshop location.

GetCROsBySerialNo (ConnectID, SerialNo: String; OutstandingOnly: Boolean)

GET: ttp(s)://hostname:port/datasnap/rest/FinconAPI/"GetCROsBySerialNo"/ConnectID/SerialNo /OutstandingOnly/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

CROType can be blank to include workshop jobs and support calls, or 'J' or 'S' to include just jobs or just support calls.

OutstandingOnly: Select to only include outstanding customer repair orders.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","CROrders":[{CRORecord,"CROrderDetail":[{CROItemRecord}], "CROrderLog":[{CROLogRecord}],"CROrderNotes":[{CRONotesRecord}],"CROrderStock":[{CROStockRecord}]}]}

if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of CRORecord structures that include arrays of CROItemRecord, CROLogRecord, CRONotesRecord and CROStockRecord structures (Keys: CROrderDetail, CROrderLog, CROrderNotes and CROrderStock) on each transaction record.

CROLocationRecord:

Key	Type	Len	Dec	Values	Default	Description
LocNo	C	6	0			Workshop Location Number
Description	C	20	0			Workshop Location Description
Active	C	1	0			Active

GetWorkshopLocations (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetWorkshopLocations"/ConnectID/

The result received is as follows:

{"Count":1,"ErrorInfo":"," "Locations": [{CROLocationRecord}]} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

WorkshopPartsOrderRecord:

Key	Type	Len	Dec	Values	Default	Description
OrderNo	C	8	0			Workshop Parts Order Number
LocNo	C	6	0			Stock location
OrderDate	D	8	0			Order Date
DateRequired	D	8	0			Date Required
CroNo	C	8	0			Customer Repair Order Number
Instruc1	C	40	0			Instructions Line 1
Instruc2	C	40	0			Instructions Line 2
Instruc3	C	40	0			Instructions Line 3
Instruc4	C	40	0			Instructions Line 4
Instruc5	C	40	0			Instructions Line 5
Instruc6	C	40	0			Instructions Line 6
Status	C	0	0			Sales Order Status

WorkshopPartsOrderDetailRecord:

Key	Type	Len	Dec	Values	Default	Description
Entry	C	1	0			Entry Type
DocNo	C	8	0			Document Number
ItemNo	C	25	0			Item Number
Quantity	N	18	2			Item Quantity
Description	C	40	0			Item Description

CreateWorkshopPartsOrder (ConnectID: String; PartsOrderRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateWorkshopPartsOrder"/ConnectID/
With `{WorshopPartsOrderRecord,"PartsOrderDetail":[{WorshopPartsOrderDetailRecord}]}` **in the body.**

PartsOrderRecord is a combination of the WorkshopPartsOrderRecord and the WorkshopPartsOrderDetailRecord structures listed above:

{WorkshopPartsOrderRecord, "PartsOrderDetail":[{WorkshopPartsOrderDetailRecord}]}
OrderNo and Status are read-only fields. OrderDate will always be the current date. All the other values will be filled with the default values where applicable. **The maximum number of detail lines is currently 1000.**

The result received is as follows:

{"ErrorInfo":"","PartsOrderInfo":{WorkshopPartsOrderRecord, "PartsOrderDetail":[{WorkshopPartsOrderDetailRecord}]}} if successful or

{"ErrorInfo":"Invalid customer repair order number or whatever the error may be."} if unsuccessful.

17: Returned Merchandise Authorizations

RMARecord:

Key	Type	Len	Dec	Values	Default	Description
RMANo	C	8	0			RMA Number
AccNo	C	6	0			Account Number
RMADate	D	8	0			RMA Date
TotalExcl	N	18	2			Total Excl
CurrencyCode	C	2	0			Currency Code
ExchangeRate	N	18	7			Quotation Exchange Rate
CustomerRef	C	20	0			Customer Reference
DebName	C	40	0			Customer Name
Addr1	C	40	0			Address Line 1
Addr2	C	40	0			Address Line 2
Addr3	C	40	0			Address Line 3
PCode	C	8	0			Postal Code
Message1	C	40	0			Message Line 1
Message2	C	40	0			Message Line 2
Message3	C	40	0			Message Line 3
RepCode	C	3	0			Sales Rep Code
Contact	C	30	0			Contact Person
TelNo	C	25	0			Telephone Number

RMADetailRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Reference	C	15	0			Any Applicable Reference
Quantity	N	18	2			Item Quantity
UnitPrice	N	18	2			Unit Price Excl
LabourPerUnit	N	18	2			Labour Cost

CreateRMA (ConnectID: String; MerchandiseRecord: TJSONValue)

POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"CreateRMA"/ConnectID/
With `{RMARecord,"RMADetail":[{RMADetailRecord}]}` **in the body.**

MerchandiseRecord is a combination of the RMARecord and RMADetailRecord structures listed above:

{RMARecord, "RMADetail":[{RMARecord}]}

RMANo is a read-only field. RMADate will always be the current date. All the other values will be filled with the default values or information retrieved from the debtor account where applicable. **Only local currency debtor accounts are allowed. The maximum number of detail lines is 100.**

The result received is as follows:

{"ErrorInfo":"","RMAInfo":{RMARecord,"RMADetail":[{RMADetailRecord}]}} if successful or

{"ErrorInfo":"Invalid account number or whatever the error may be."} if unsuccessful.

RMAQuantityResultRecord:

Key	Type	Len	Dec	Values	Default	Description
AccNo	C	6	0			Account Number
ItemNo	C	25	0			Item Number
Sold	N	18	2			Quantity Sold
Credited	N	18	2			Quantity Credited

GetRMAQuantity (ConnectID, AccNo, ItemNo, FromDate, ToDate: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetRMAQuantity"/ConnectID/AccNo/ItemNo /FromDate/ToDate/

The result received is as follows:

{"ErrorInfo":"","RMAInfo":{RMAQuantityResultRecord}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

NOTE:

- **Sold** is the total invoice quantity for the selected date range minus credit notes issued for said invoices.
- **Credited** is the total credit note quantity for the selected date range of credit notes that weren't issued for the above-mentioned invoices. In other words, these credit notes fall in the selected date range, but the applicable invoices don't.

18: Documents

UploadDocumentURLRecord:

Key	Type	Len	Dec	Values	Default	Description
DocType	C	1	0			Document Type
DocNo	C	8	0			Document Number
URL	C	200	0			Document URL

UploadDocumentURL (ConnectID, UploadDocumentURLRecord: TJSONValue)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"UploadDocumentURL"/ConnectID/
With `{UploadDocumentURLRecord}` **in the body.**

DocType: The document type. Only invoices are currently allowed, so DocType must always be 'I'.

DocNo: The document number. The document number will automatically be left padded with zeros up to the required length.

URL: The URL string.

The result received is as follows:

{"ErrorInfo":"","DocumentInfo":{UploadDocumentURLRecord}} if successful or

{"ErrorInfo":"Whatever the error may be."} if unsuccessful.

DocumentPdfResultRecord:

Key	Type	Len	Dec	Values	Default	Description
DocType	C	1	0			Document Type
DocNo	C	8	0			Document Number
DocumentPdf	C	-	0			Pdf Document

The DocumentPdf value is a base64 encoded string of the generated Pdf file.

GetDocumentPdf (ConnectID, DocType, DocNo, LayoutName: String; NoImages: Boolean)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetDocumentPdf"/ConnectID/DocType/DocNo /LayoutName/NoImages/

DocType: The document type is **case sensitive** and can be any one of the following:

Document	DocType
Debit Note (Branch)	X
Debit Note (Head Office)	V
Debit Note (Import)	E
Debit Note (Production)	p
Debit Note (Sub-Assembly)	h
Debit Note (Supplier)	D
Debtor Credit Note	C
Debtor Invoice	I

Faulty Stock (Receive from Supplier)	R
Faulty Stock (Send to Supplier)	F
Goods Received Voucher (Branch)	T
Goods Received Voucher (Head Office)	U
Goods Received Voucher (Import)	M
Goods Received Voucher (Production)	o
Goods Received Voucher (Sub-Assembly)	g
Goods Received Voucher (Supplier)	G
Serial Number Swop-out	W
Service Received Voucher	A
Service Received Voucher - Buyout	i
Stock Adjustment (General)	N
Stock Adjustment (Consignment Stock)	v
Stock Adjustment (Normal Stock)	z
Stock Adjustment (Stock at Suppliers)	u
Stock Adjustment (Technician Stock)	w
Stock Adjustment (WIP Stock)	x
Stock Transfer (General)	Z
Stock Transfer (Consignment Stock)	n
Stock Transfer (Work In Progress)	m

LayoutName: Enter the required layout name. The default layout for the document will be use if LayoutName is blank or invalid. **Text layouts cannot be used.**

NoImages: Select to exclude images on the layout from the generated pdf string in order to reduce the size.

The result received is as follows:

{"ErrorInfo":"","DocumentInfo":{DocumentPdfResultRecord}} if successful or

{"ErrorInfo":"Document not found or whatever the error may be."} if unsuccessful.

19: API Functions

KeepAlive (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”KeepAlive”/ConnectID/

The purpose of this function is to prevent a ConnectID from being reassigned as the result of a timeout. If you are going to be inactive while logged in for a period longer than the defined user time-out, you need to keep the connection alive.

The result received is as follows:

{"Successful":true,"ErrorInfo":""} if successful or

{"Successful":false, "ErrorInfo":"Whatever the error may be"} if unsuccessful.

GetSessionInfo (ConnectID: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetSessionInfo”/ConnectID/

The result received is as follows:

{"MaxConfig":30,"InUse":1,"MaxUsed":1,"ErrorInfo":""}

KillUsers (ConnectID: String; All: Boolean)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/”KillUsers”/ConnectID/All/

This function can be called periodically with All = false to initialise user slots that haven’t being used for a period longer than the user timeout. Note that even when All is set to true, the API will still complete current requests. Any subsequent requests will require a login though.

The result received is as follows:

{"Successful":true,"ErrorInfo":""} if successful or

{"Successful":false, "ErrorInfo":"Whatever the error may be"} if unsuccessful.

20: Functions for Mobile Applications

API access levels needed:

Stock Items: 1
Customer Repair Orders: 2
Debtors: 3
Quotations: 1
Documents: 2

Retrieving and updating Customer Repair Orders:

MobileStockRecord:

Key	Type	Len	Dec	Values	Default	Description
ItemNo	C	25	0			Item Number
Description	C	40	0			Item Description
UnitOfMeasure	C	2	0			Unit Of Measure
Category	C	3	0			Item Category
Group	C	6	0			Item Group

GetStockMobile (ConnectID, FilterList, FilterType: String; RecNo, Count, ArrPos: Integer)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetStockMobile"/ConnectID/FilterList/ FilterType/RecNo/Count/ArrPos

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

FilterList represents a list of the categories or groups separated by a comma of items to be retrieved.

FilterType can be C if the filter list contains a list of categories or G if the filter list contains a list of groups.

ArrPos: This value must be 0 for the first call and then the returned value on any subsequent calls.

The result received is as follows:

{"Count":1,"RecNo":88,"ArrPos":1,"ErrorInfo":"","Stock":[{MobileStockRecord}]} if successful or
{"Count":0,"RecNo":0, "ArrPos":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

GetCROListMobile (ConnectID, TechCode, OrdType: String; RecNo, Count: Integer)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCROListMobile"/ConnectID/TechCode /OrdType/RecNo/Count/

Refer to the section on **General principles and list type functions** for an explanation of the parameters and the return values.

OrdType: J = Customer Repair Orders, S = Support Calls, Blank = All.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","CROrders":[{CRORecord,"CROrderDetail":[{CROItemRecord}]}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of CRORecord that includes an array of CROItemRecord on each transaction record.

GetCROInfoMobile (ConnectID, CRONo, SerialNo: String)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetCROInfoMobile"/ConnectID/CRONo/SerialNo/

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","CanClose":"true/false","ItemCount":1,"CROrders":[{CRORecord, "CROrderDetail":[{CROItemRecord}],"CROrderLog":[{CROLogRecord}],"CROrderNotes":[{CRONotesRecord}], "CROrderStock":[{CROStockRecord}], "Visit1":{CROVisitRecord},"Visit2":{CROVisitRecord}, "Visit3":{CROVisitRecord}, "Visit4":{CROVisitRecord},"Visit5":{CROVisitRecord}}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

"CanClose" indicates if the customer repair order is in a state that it's status can be changed to *Closed*. "ItemCount" indicates if the number of items on the repair order – ItemCount will be -1 if it is a support call. The transactions are returned in an array (with only one element in this case) of CRORecord that includes arrays of CROItemRecord, CROLogRecord, CRONotesRecord and CROStockRecord structures (Keys: CROrderDetail, CROrderLog, CROrderNotes and CROrderStock) and the records for visits on each transaction record.

MobileCROUpdateRecord:

Key	Type	Len	Dec	Values	Default	Description
TechCode	C	3	0			Technician Code
CRONote	C	180	0			CRO Note
Status	C	1	0			Item Status*
Diagnosis	C	300	0			Item Diagnosis
WorkDone	C	300	0			Item Work Done
Rejected	C	300	0			Repair Rejected Reason
CompletionDate	D	8	0			Date Completed
Visit1 - Visit5						CROVisitRecord

IMPORTANT: Only include the fields to be updated.

*Status values consist of one character where:

- A blank = Cancelled (CRO Status only)
- P = Pending
- D = Technician started
- C = Waiting for customer
- S = Waiting for spares
- A = Sent away
- W = Supplier waiting for reply
- R = Received back
- H = Technician finished
- F = Completed
- V = Approved (CRO Status only)
- I = Invoiced (CRO Status only)

- T = Closed (CRO Status only)

TechCode and **CRONote:** To add a CRO Note.

Status: The new item status can be P, D, C, S, H or F if the current item status is not A or W. For support calls, the new item status can only be P or F.

CROInfoRecord:

Key	Type	Len	Dec	Values	Default	Description
CRONo	C	8	0			CRO Number
SerialNo	C	40	0			Serial Number
ItemCount	I					Number of items on CRO
Changed	L	1				Information Changed
CROStatus	C	1	0			CRO Status
CanClose	L	1	0			CRO status can change to *Closed*
Status	C	1	0			Item Status
Diagnosis	C	300	0			Item Diagnosis
WorkDone	C	300	0			Item Work Done
Rejected	C	300	0			Repair Rejected Reason
CompletionDate	D	8	0			Date Completed
Visit1 - Visit5						CROVisitRecord

UpdateCROMobile (ConnectID, CRONo, SerialNo: String; MobileCROUpdateRecord: TJSONObject)
POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"UpdateCROMobile"/ConnectID/CRONo/SerialNo/
With `{MobileCROUpdateRecord}` **in the body.**

The information returned is as follows:
{"ErrorInfo":"Error Description") if unsuccessful or
{"ErrorInfo":"", "CROrderNotes":[{CRONotesRecord}],"RepairOrderInfo",{CROInfoRecord}} if successful

CROrderNotes will only be included if a CRO note was added.

RepairOrderInfo will only be included if the item status, diagnosis, work done, rejected reason, completion date or one or more of the visits were updated. Only the visits included in the request will be returned.

CloseCROMobile (ConnectID, CRONo: String)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"CloseCROMobile"/ConnectID/CRONo/

CRONo: The customer repair order that has to be set to closed.

The information returned is as follows:
{"ErrorInfo":"Error Description") if unsuccessful or
{"ErrorInfo":"", "CROStatus":"Blank, F, V, I or T - Refer to the status list above" } if successful

The circumstances under which a Customer Repair Order can be closed, are as follows:

1. *CROStatus* is F, *Approve CROs* (Fincon setting) is disabled, there are no stock items or charges to be invoiced and there are no outstanding parts orders.
2. *CROStatus* is V, there are no stock items or charges to be invoiced and there are no outstanding parts orders.
3. *CROStatus* is I and there are no outstanding parts orders. If there are any, they must be cancelled first.

Retrieving document lists for signatures:

GetInvoicesMobile (ConnectID, AccNo, MinDate, MaxDate, LocNo, RepCode: String; ExSigned, ExNoSign, ExMonthly: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetInvoicesMobile"/ConnectID/AccNo /MinDate/MaxDate/LocNo/RepCode/ExSigned/ExNoSign/ExMonthly/RecNo/Count/

LocNo can contain a single stock location (for example 0) or a list of locations separated by commas (for example 0,1,3).

ExSigned: Set to true to exclude entries where signed documents are already present.

ExNoSign: Set to true to exclude entries where signed documents are not required.

ExMontly: Set to true to exclude monthly invoices.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","DebtorTransactions":[{DebtorTransactionRecord}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of DebtorTransactionRecord that includes *"HasSignedDoc":"Y/N"* and *"MustSignDoc":"Y/N"* on each transaction record - refer to the section on Debtors for a definition of DebtorTransactionRecord.

GetQuotesMobile (ConnectID, AccNo, MinDate, MaxDate, LocNo, RepCode, StatusList: String; ExSigned, ExNoSign: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuotesMobile"/ConnectID/AccNo /MinDate/MaxDate/LocNo/RepCode/StatusList/ExSigned/ExNoSign/RecNo/Count/

LocNo can contain a single stock location (for example 0) or a list of locations separated by commas (for example 0,1,3).

StatusList: Specify a list of quotation statuses to be included. Valid values include:

" " (Blank Character): Active quotations

"O": Quotations with a sales order present

ExSigned: Set to true to exclude entries where signed documents are already present.

ExNoSign: Set to true to exclude entries where signed documents are not required.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","Quotations":[{QuotationRecord}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of QuotationRecord that includes*"CurrAbbr":"Currency abbreviation as defined in Fincon", "HasSignedDoc":"Y/N"* and *"MustSignDoc":"Y/N"* on each transaction record - refer to the section on Quotations for a definition of QuotationRecord.

MobileTransferRecord:

Key	Type	Len	Dec	Values	Default	Description
Entry	C	1	0			Entry Type
DocNo	C	8	0			Document Number
Date	D	8	0			Transaction Date
AccNo	C	6	0			Account Number
DebName	C	40	0			Debtor Name
Location	C	6	0			Stock Location

GetTransfersMobile (ConnectID, AccNo, MinDate, MaxDate, LocNo: String; ExSigned, ExNoSign: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetTransfersMobile"/ConnectID/AccNo /MinDate/MaxDate/LocNo/ExSigned/ExNoSign/RecNo/Count/

LocNo can contain a single stock location (for example 0) or a list of locations separated by commas (for example 0,1,3).

ExSigned: Set to true to exclude entries where signed documents are already present.

ExNoSign: Set to true to exclude entries where signed documents are not required.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","Transfers":[{MobileTransferRecord}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of MobileTransferRecord that includes *"HasSignedDoc":"Y/N"* and *"MustSignDoc":"Y/N"* on each transaction record.

GetRepairOrdersMobile (ConnectID, AccNo, MinDate, MaxDate, LocList, TechCode, StatusList: String; ExSigned, ExNoSign: Boolean; RecNo, Count: Integer)

GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetRepairOrdersMobile"/ConnectID/AccNo /MinDate/MaxDate/LocList/TechCode/StatusList/ExSigned/ExNoSign/RecNo/Count/

LocList can contain a single work shop location (for example HO) or a list of locations separated by commas (for example HO,JHB,CPT).

StatusList: Specify a list of sales order statuses to be included. Valid values include:

"F": Completed customer repair orders

"V": Approved customer repair orders

"I": Invoiced customer repair orders

"T": Closed customer repair orders

ExSigned: Set to true to exclude entries where signed documents are already present.

ExNoSign: Set to true to exclude entries where signed documents are not required.

The result received is as follows:

{"Count":1,"RecNo":1,"ErrorInfo":"","CROrders":[{CRORecord}]}
if successful or

{"Count":0,"RecNo":0,"ErrorInfo":"No records found or whatever the error may be."} if unsuccessful.

The transactions are returned in an array of CRORecord that includes *"HasSignedDoc":"Y/N"* and *"MustSignDoc":"Y/N"* on each transaction record - refer to the section on Customer Repair Orders for a definition of CRORecord.

Retrieving document info for signatures:

GetInvoiceInfoMobile (ConnectID, DocNo: String)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetInvoiceInfoMobile"/ConnectID/DocNo/

GetQuoteInfoMobile (ConnectID, DocNo: String)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuoteInfoMobile"/ConnectID/DocNo/

GetTransferInfoMobile (ConnectID, DocNo: String)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetSalesOrderInfoMobile"/ConnectID/DocNo/

GetRepairOrderInfoMobile (ConnectID, DocNo: String)
GET: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetRepairOrderInfoMobile"/ConnectID/DocNo/

These functions return all the information needed for displaying the applicable contents. Refer to **FinconRESTClient** with regards to the information returned.

GetInvoiceInfoMobile can also return some of the following settings for print options depending on the account's settings:

"InvoiceLayout": The preferred layout to be used
"StkProp":"Y/N": Include stock properties (default N)
"SerialNo":"Y/N": Include serial numbers (default N)
"AccItemNo":"Y/N": Print accounts item numbers (default N) **
"OrgItemNo":"Y/N": Print originating item numbers (default N) **
"AltItemDes":"Y/N": Print alternate description (default N)
"LineTotals":"Y": Print line totals (default setting) **
"GroupTotals":"Y": Print group totals **
"TotalsOnly":"Y": Print totals only **
"GroupDesc":"Y": Print group descriptions **
"GroupDescSpecs":"Y": Print group descriptions and specifications **
"GroupDescDetSpecs":"Y": Print group descriptions and detailed specifications **
**: These settings are mutually exclusive.

NOTE: These print settings (except for Include Serial Numbers) are also applicable to quotations when requesting a pdf document - refer to *SignatureRecord* below.

Adding signatures to documents:

SignatureRecord:

Key	Type	Len	Dec	Values	Default	Description
Signature	C					Customer Signature
ReceivedBy	C	30				Customer Name
Latitude	C	12	0			Latitude (Includes 6 decimals)
Longitude	C	12	0			Longitude (Includes 6 decimals)
DeviceName	C	20	0			Device Name
PrintSettings						Refer to ***GetInvoiceMobile*** above

Signature: The signature must be a base64 encoded string of the image in Portable Network Graphic (png) format. The signature image will be resized to fit inside the designated area on the layout, while retaining its aspect ratio. It is therefore imperative to remove any white space around the physical signature to avoid it from being reduced to a minute size.

ReceivedBy, Latitude, Longitude and DeviceName: This information, together with the request date and time is added with the signature to the document. If a signature is present, it will be added as defined on the layout or as an extra page to the delivery note if the signature area on the layout is not defined. If *Signature* is blank or if the image cannot be loaded, none of these fields will be added.

GetInvoicePdfMobile (ConnectID, DocNo, LayoutName: String; SignatureRecord: TJSONObject)
POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetInvoicePdfMobile"/ConnectID/DocNo/ LayoutName/
With {SignatureRecord} **in the body.**

The information returned is as follows:
{"ErrorInfo":"Error Description") if unsuccessful or
{"ErrorInfo":"", "DocumentPdf":"The base64 encoded string of the signed document in pdf format"} if successful

LayoutName: If the layout name is blank or invalid, the default layout will be used if specified.

GetQuotePdfMobile (ConnectID, DocNo, LayoutName: String; SignatureRecord: TJSONObject)
POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetQuotePdfMobile"/ConnectID/DocNo/ LayoutName/
With {SignatureRecord} **in the body.**

The information returned is as follows:
{"ErrorInfo":"Error Description") if unsuccessful or
{"ErrorInfo":"", "DocumentPdf":"The base64 encoded string of the signed document in pdf format"} if successful

LayoutName: If the layout name is blank or invalid, the default layout will be used if specified.

GetTransferPdfMobile (ConnectID, DocNo, LayoutName: String; SignatureRecord: TJSONObject)
POST: http(s)://hostname:port/datasnap/rest/FinconAPI/"GetTransferPdfMobile"/ConnectID/DocNo/ LayoutName/
With {SignatureRecord} **in the body.**

The information returned is as follows:

{“ErrorInfo”:”Error Description”) if unsuccessful or
{“ErrorInfo”:””, “DocumentPdf”:”The base64 encoded string of the signed document in pdf format”} if successful

LayoutName: If the layout name is blank or invalid, the default layout will be used if specified.

GetCRODelNotePdfMobile (ConnectID, CRONo, SerialNo, LayoutName: String; SignatureRecord: TJSONObject)
POST: http(s)://hostname:port/datasnap/rest/FinconAPI/”GetCRODelNotePdfMobile”/ConnectID/CRONo/ SerialNo/LayoutName/
With `{SignatureRecord}` **in the body.**

The information returned is as follows:
{“ErrorInfo”:”Error Description”) if unsuccessful or
{“ErrorInfo”:””, “DocumentPdf”:”The base64 encoded string of the signed document in pdf format”} if successful

LayoutName: If the layout name is blank or invalid, the default layout will be used if specified.

NOTES: A Workshop Delivery Note cannot be requested for a support call. The serial number is not required if the Delivery Note is applicable to all the items on the CRO, provided that the CRO’s status is F, V, I or T.